A Guide to Creature Animations for Infinity Engine Games

by The Artisan

This guide will provide a step-by-step guide to importing custom 3D animations into Blender, rendering said animations into individual frames, and using GIMP to convert them into indexed images, ready to be made into animations for the Infinity Engine games.

Before we get started, though, I should preface this by saying that I am by no means an expert of 3D modeling or the use of any of the software I'll be talking about. I barely even understood how to use most of these tools when I first started. This process was done through trial-and-error, and designed to be specifically as simple as I could possibly make it. Creating animations for IE games is obscure and tedious, and there is no way around it. But I hope this guide at least allows for those truly dedicated towards adding new content to have some direction.

Getting Started

In order to follow the process of making animations, I have used the following programs with links, along with associated plugins that provide necessary or otherwise helpful functions

- Blender (I have used v4.0+ for this, but older versions as far back as v3.0 should also be fine)
    - Neverblender for NWN1 animations
    - Blender add-on for Neverwinter Nights 2 – I will not be detailing how to render NWN2 animations in this guide since the process is a little different, but it's an option if you want to try it
    - Palette Studio which provides helpful tools for cameras and rendering
- GIMP (any version should be fine)
    - Export Layers (this allows us to export each frame of an animation as its own file after changes – nobody wants to do this manually)

    - For newer versions, GIMP 3.0.0 with Batcher may be a more up to date alternative, though I have not tried it. If unsure, just stick with the above.

- NearInfinity
- NWN Explorer
- BAM Resizer

Animation File Format

Formatting for IE creature animations is messy and complicated. There are several different animation types, several of which use their unique formats, but for the average user there is little need to distinguish them. If you are interested in the details, IESDP has several sections dedicated to breaking it down. For our purposes, we will be using the IWD-style type (E000), as it is the easiest to work with with the most comprehensive formatting. To break it down, the E000 type labels its files with specific animation sequences, which are designated as follows:

A1 – attack 1/slash

A2 – attack 2/backslash

A3 – attack 3/jab*

A4 – attack 4/shoot**

CA – cast

DE – dying

GH – getting hit/damage

GU – getting up/awake

SC – ready

SD – pause

SL – sleep***

SP – conjure

TW – dead****

WK – walk

* The attack 3/jab animation is funky with E000 in that it only works with left-facing animations. If creature is facing right or if this animation isn't present, A1 or A2 sequences are randomly used instead. Can be skipped.

** This is used for ranged attacks. Can be skipped if a ranged attack animation isn't absolutely necessary.

*** The DE (dying) animation can be used for this sequence unless a unique sleep/unconscious animation is specifically needed

**** A one to two frame animation for dead creatures. Generally the last frame of the DE (dying) animation can suffice, so we do not need to render for this specifically

Each sequence has a normal version and an 'E' – right facing – version. As an example, the naming convention for walking animations would be xxxxWK.bam and xxxxWKE.bam, for left- and right-facing animations respectively.

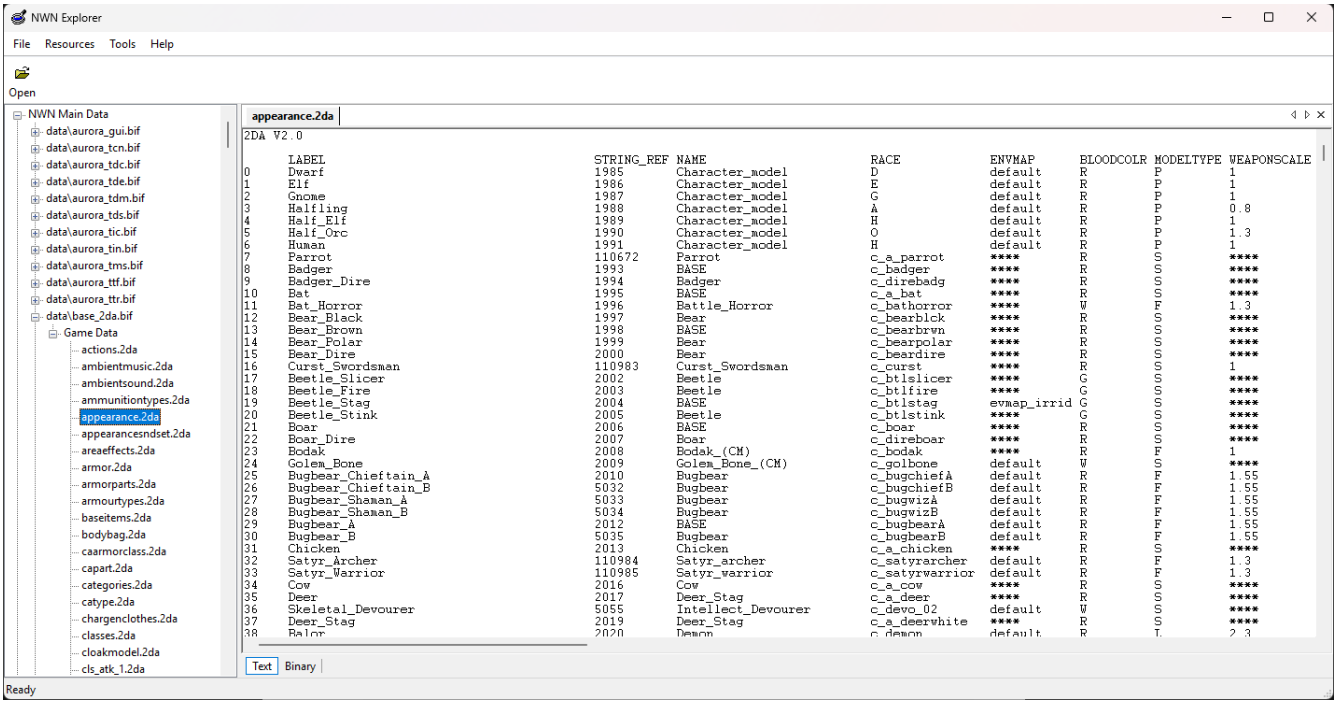
We also need a resref (resource reference) for our animation, which is the four-character reference we use in the front of our file names. In practice, modders will only have two characters to distinguish their animations with, as we will be using designated two-letter prefixes to make sure our file names don't clash with other modders. If you're an aspiring modder and haven't already signed for a prefix, check here at Gibberlings 3 for details – don't be one of those clowns who think they're too good for one, please!

Finding our creature model

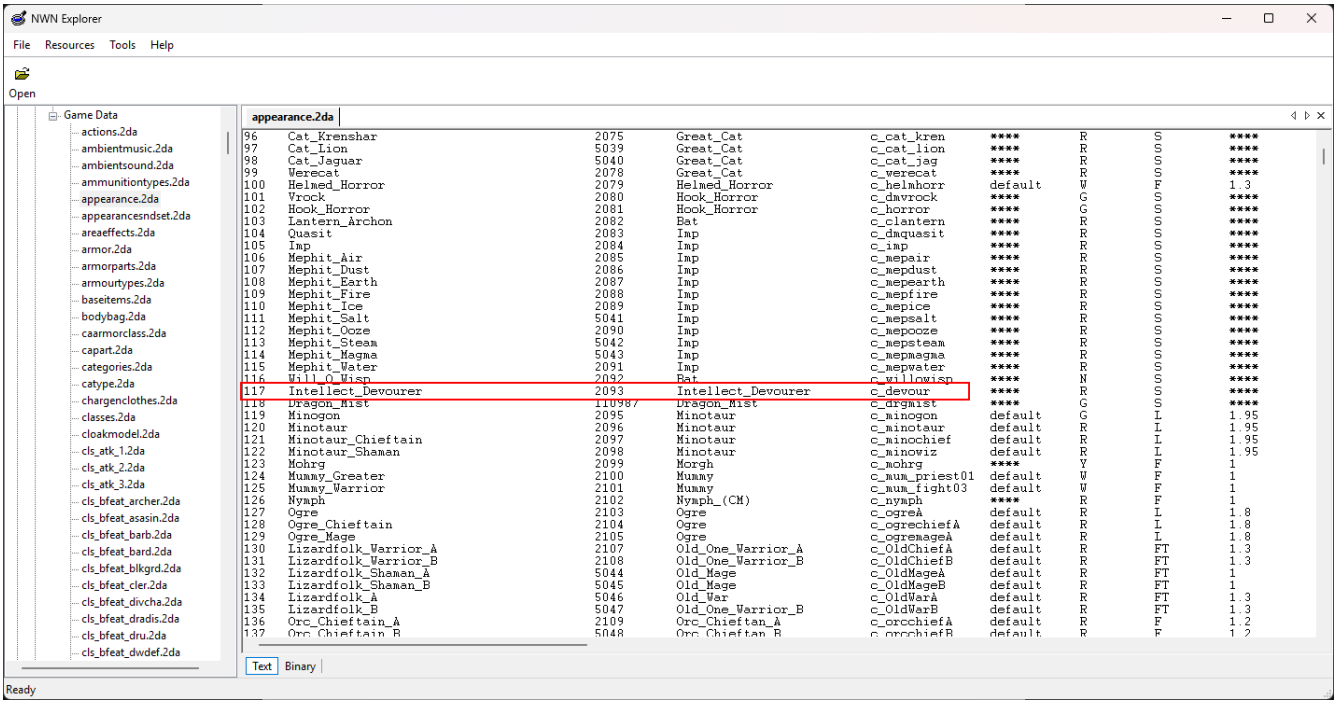
To begin with creating our animation, we first need to find an animated 3D model to use as a reference. Technically, this could be anything that you can import into Blender. If you had the talent for creating and animating 3D models with all the sequences I've mentioned above, you could even create your own original animations if you liked. For this guide though, I will be pulling animations from Neverwinter Nights, since they match decently with the game's art style, come with premade animations, and won't lead to any potential rights issues… hopefully.

For our example, let's export the Intellect Devourer animation, using NWN Explorer.

In order to find out what the necessary files to export are, though, we'll have to look through the 2da files, specifically appearance.2da, which you find under data\base_2da.bif\Game Data\:



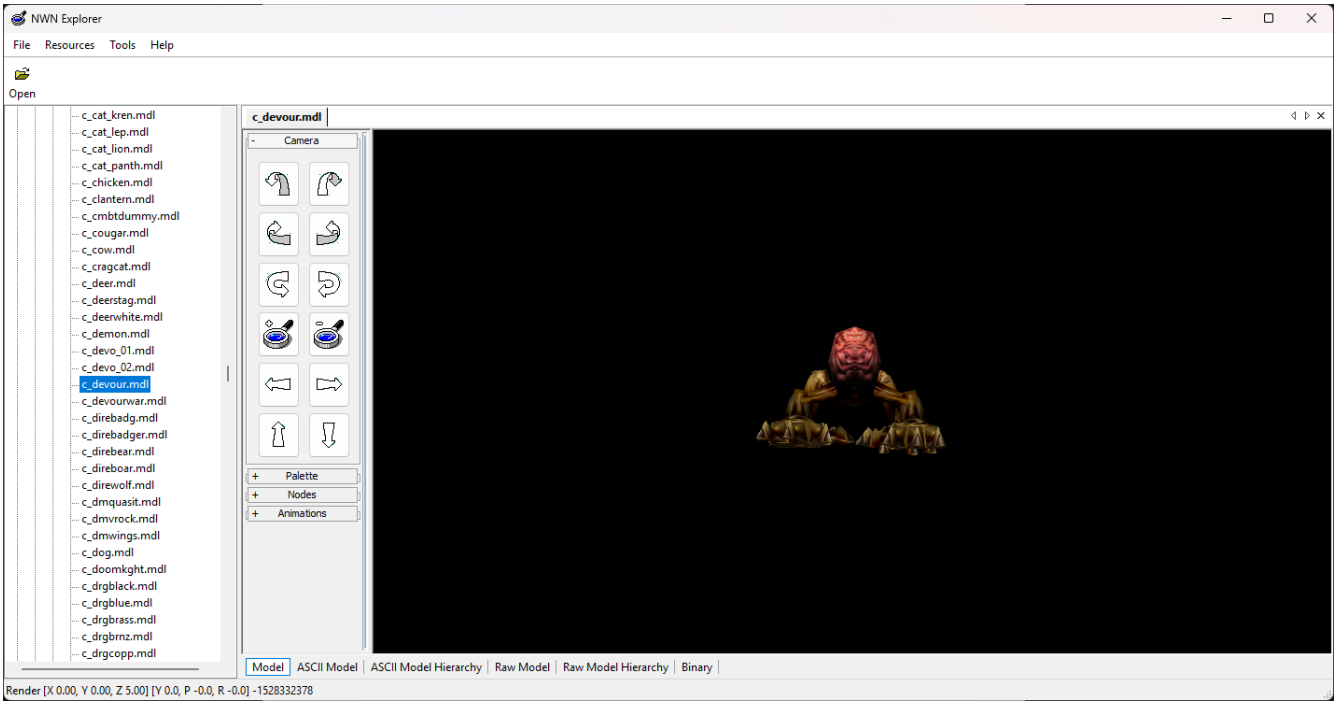
Here, we see the list of creature models in the game along with their information, but the only entry we're interested in right now is the RACE section. Let's see what it says for the Intellect Devourer:



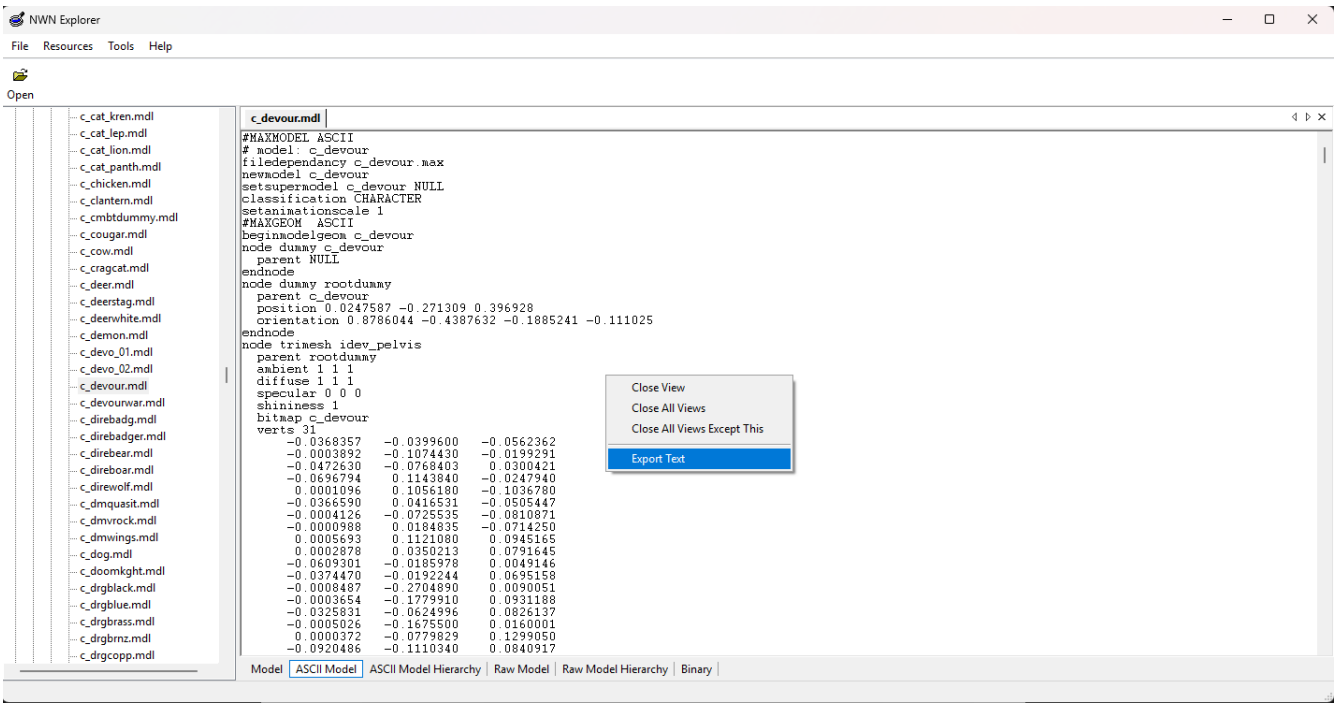
For convenience's sake, it might be a good idea to export the appearance.2da file (just right click its name on the left and choose Export) to open it up a text editor so we can use search functions through it. In any case, we can see under entry 117, the Intellect Devourer is attributed to the c_devour race, which gives us a reference for what file name to search for.

Use Ctrl+S to open up the search bar and type in c_devour.mdl, since that is specifically the .mdl (model) file we want.

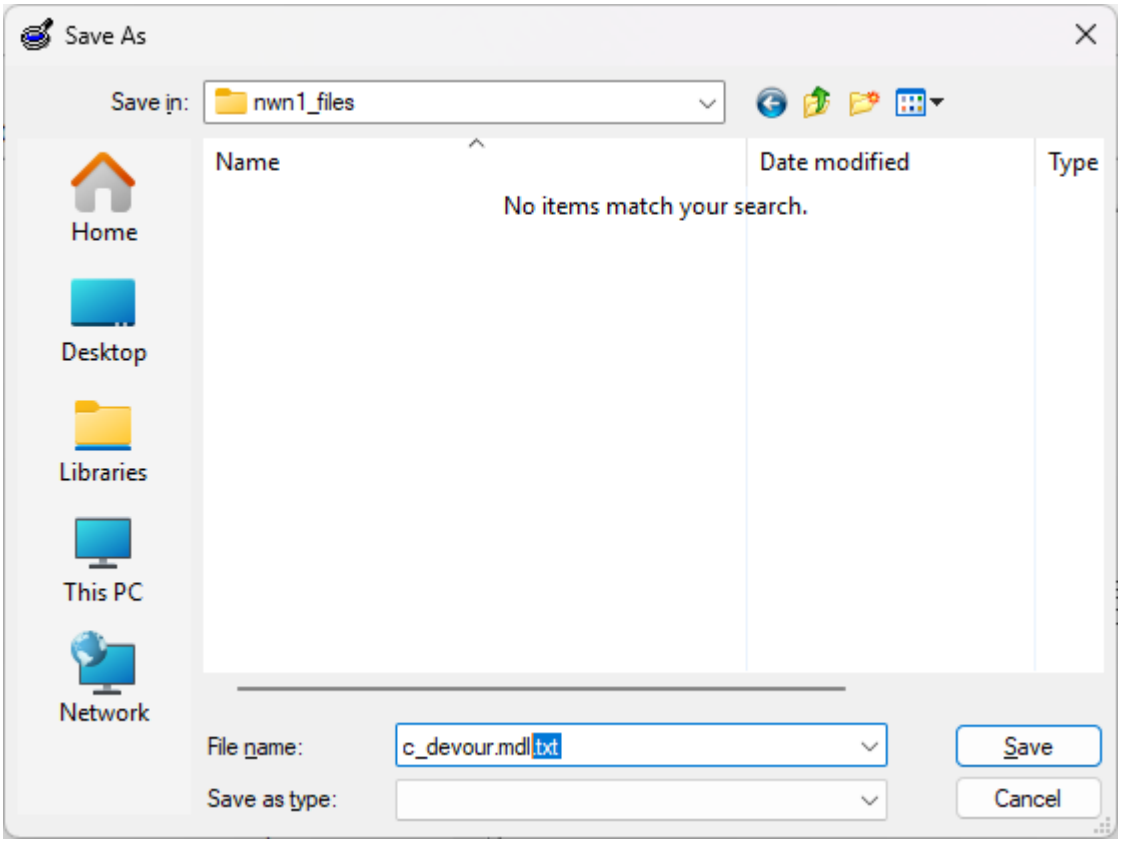
Once you've found the file, simply double-click it and you'll get a preview of the model:



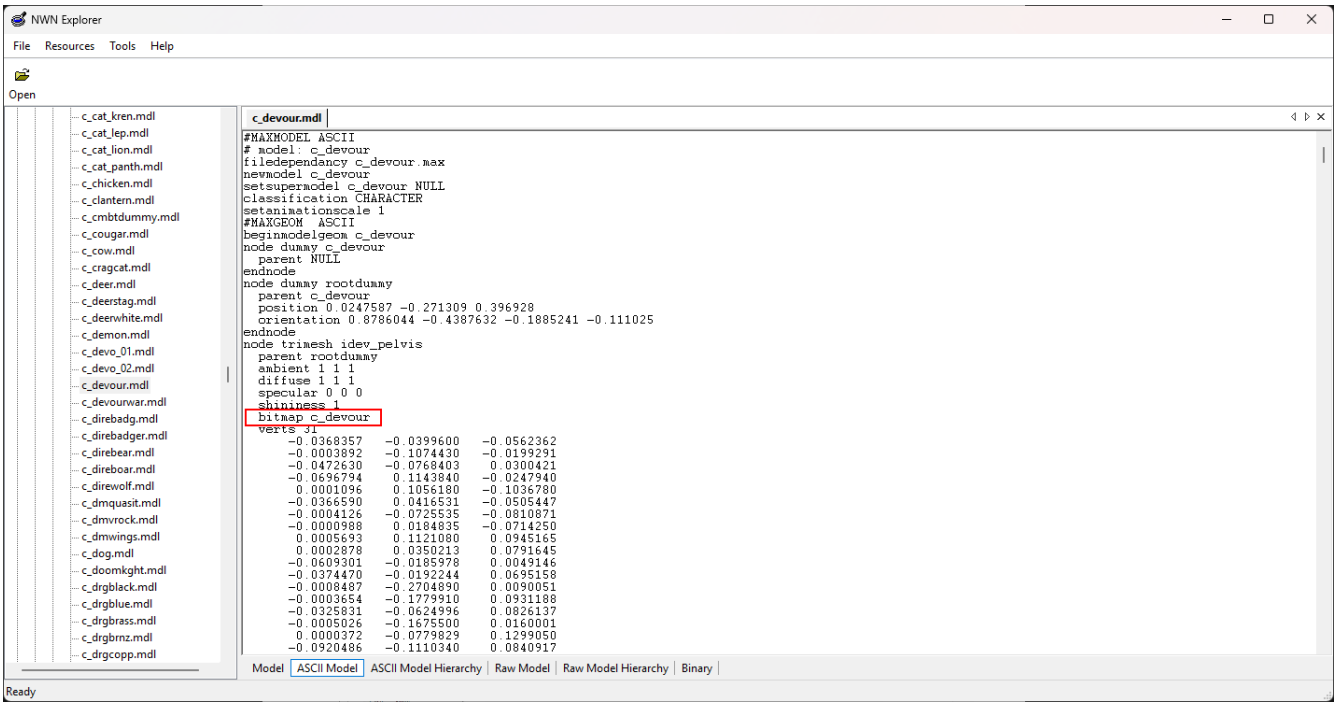
Now your immediate thought might be to Export the file as it is. DON'T do that, since the file we get won't be readable in Blender. Instead, what we actually want to do is open the ASCII Model tab. What you'll see just seems like a bunch of gibberish data. Don't worry, most of it's not important to understand. First things first, what you want to do is right-click on the data and choose Export Text:



This will give us the prompt to save the file. REMOVE THE .txt AT THE END FIRST, and then save it in a directory that's convenient for you.



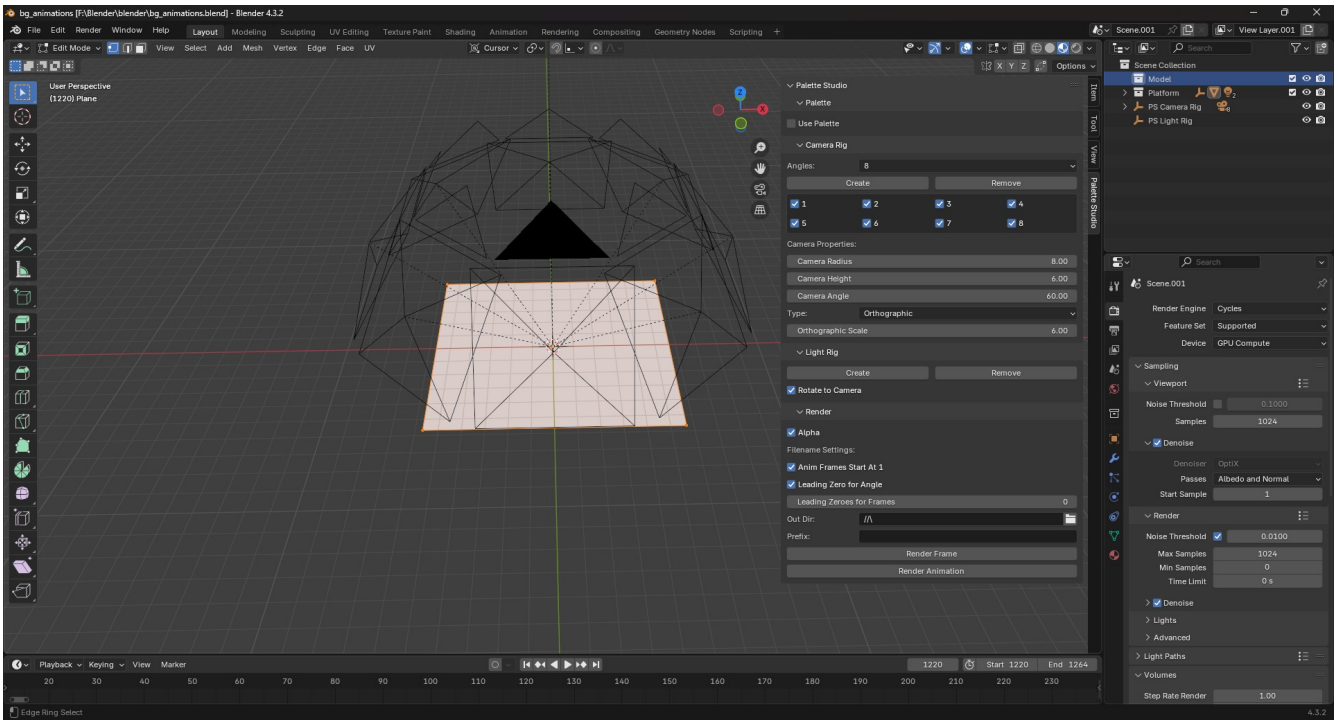
So now we have our model. But we're not done here yet. There's just a one more file we still need, that being the texture file. To find this, we could simply search for c_devour.dds, but depending on the model, the texture may not have the same name. Fortunately, the .mdl information will tell us what we need under the 'bitmap' entry.



So in this case, the texture file in our case is indeed named c_devour. Search for the file c_devour.dds and choose 'Export TGA', then save it to the same directory as you saved the .mdl file, and we have our model and texture files. Now we can start up Blender and get to rendering.

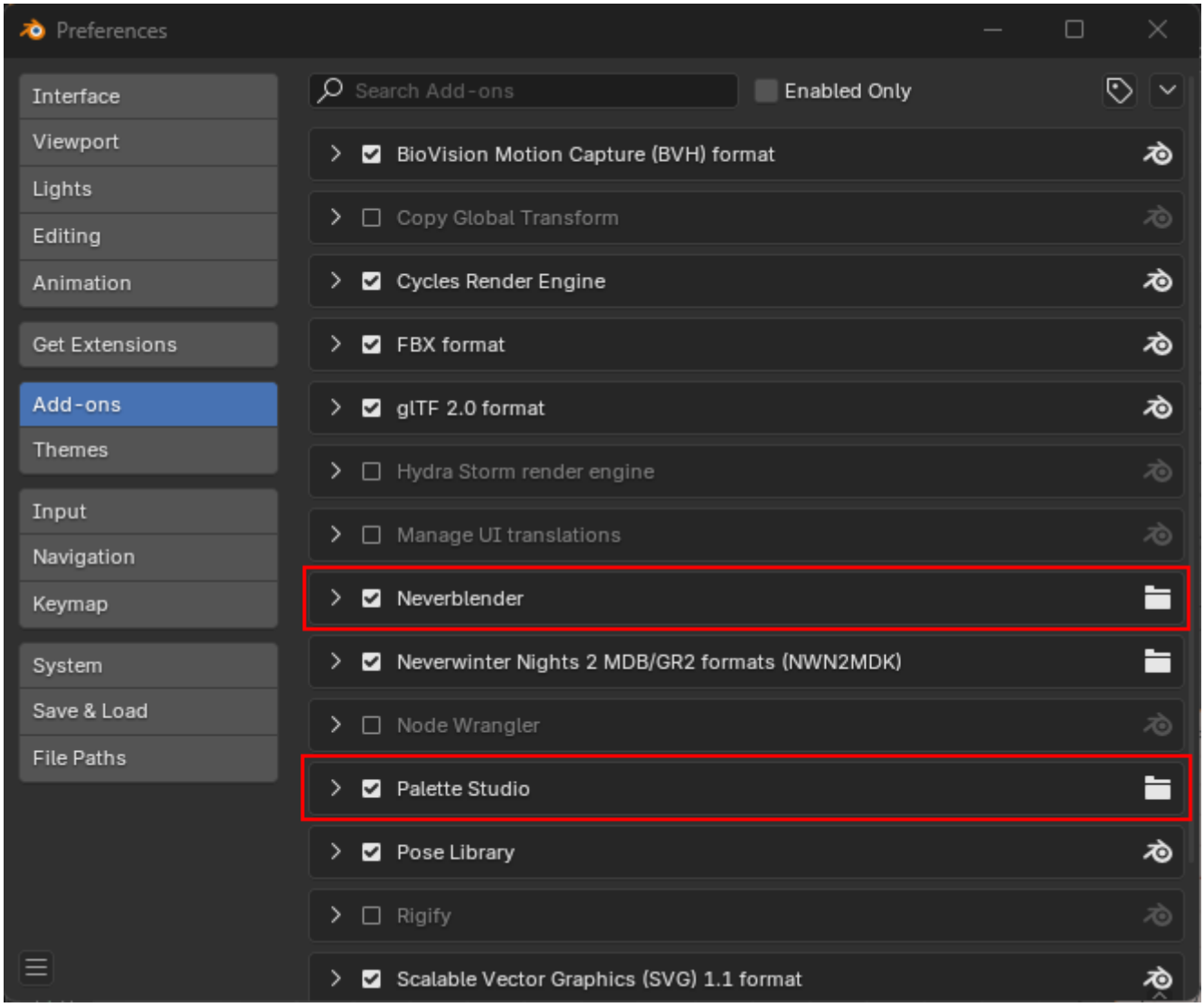
Lights, Camera, Animation

In order to render our animation in the most visible manner, we need to make an appropriate setup with lighting and cameras to display our animations in the various necessary angles. This can get a little complicated, but I've done most of the prep work so that all you need to do is import and render the model. Open up our bg_animations.blend file and you should see this:

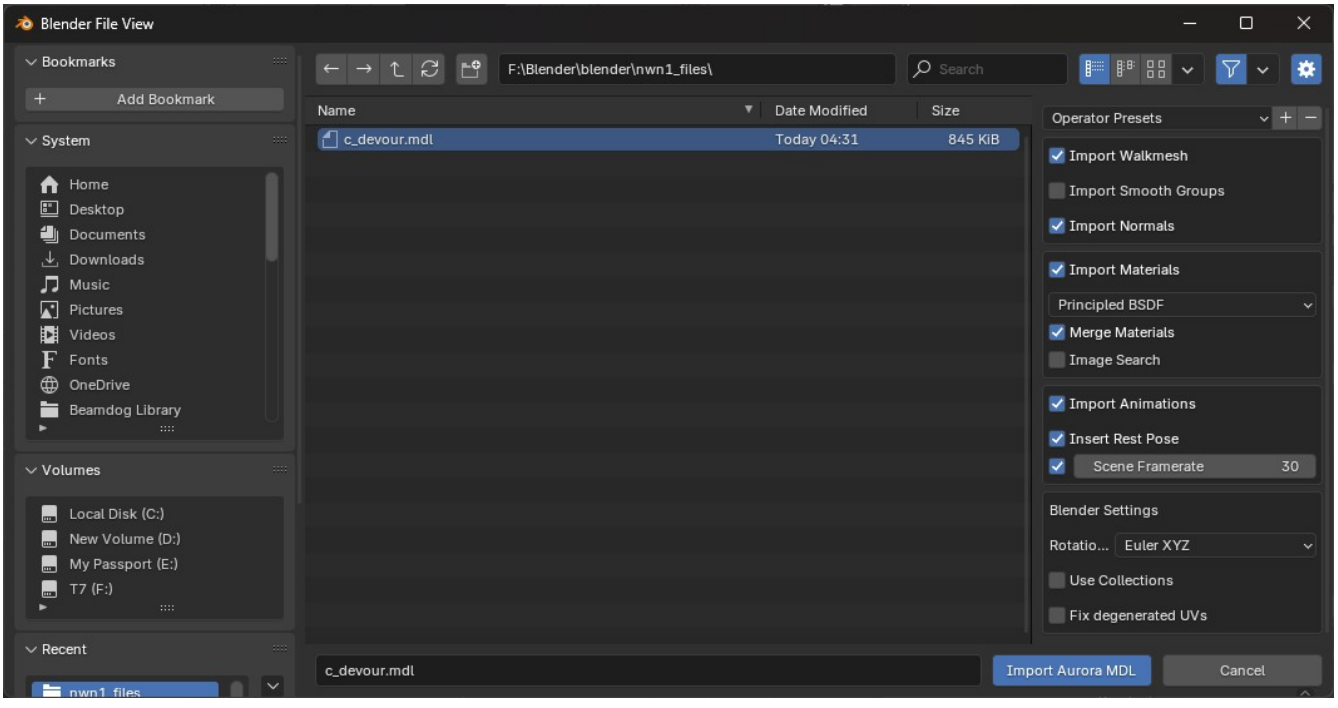


This may look very confusing for anyone who hasn't used Blender, but don't worry, it's actually very simple. What you are seeing are the cameras set on the eight different angles in which each animation will be rendered, and the white square in the center is the invisible (while you can see it here, in the final rendered image it will be fully transparent) platform on which we will set our model so that we can capture its shadow in the final render.

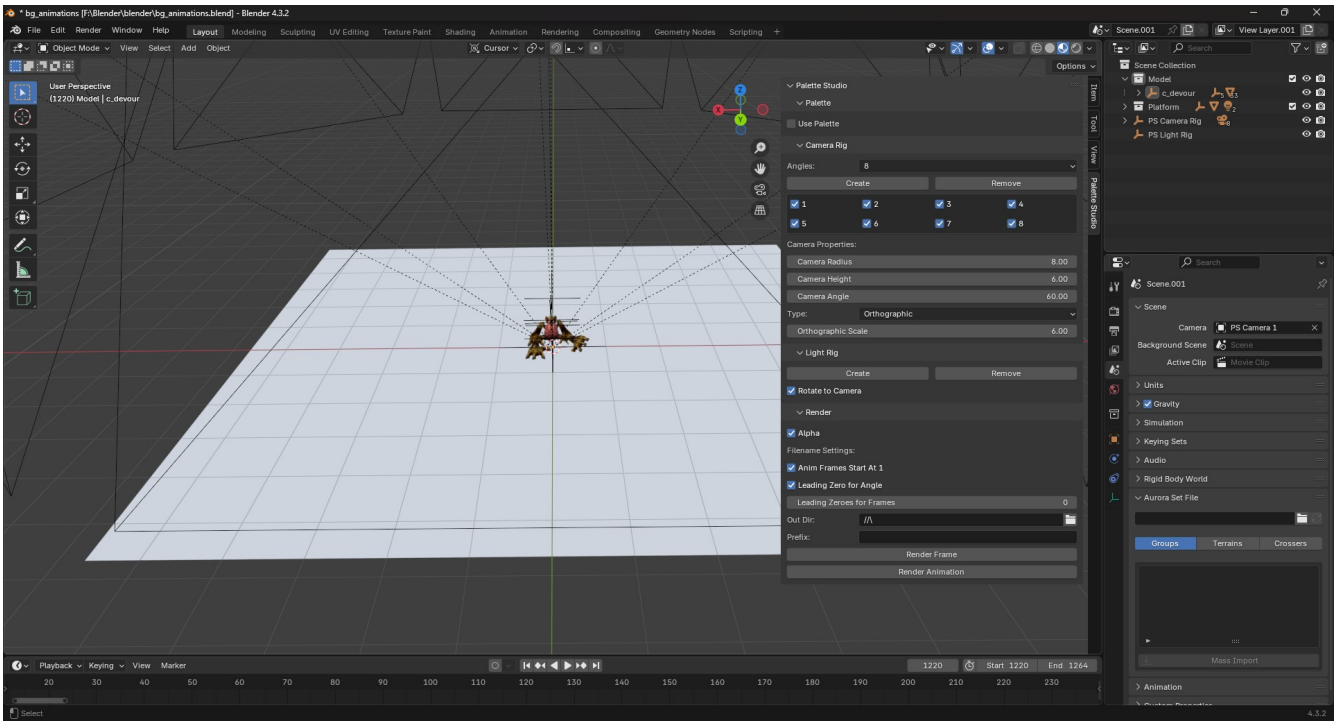
But first off, if you have not already installed the necessary add-ons (Neverblender and Palette Studio), your setup may not look the same as displayed here. If so, go to Preferences, then Add-ons, choose Install from Disk from the dropdown menu in the top right corner, and select the .zip files for the add-ons from wherever you downloaded them. Your add-on list should have these two listed and enabled:



Now we can import our NWN model. Go to File→Import, and find the .mdl file in whatever directory you saved it to previously, and make sure you have these settings as shown on the right:

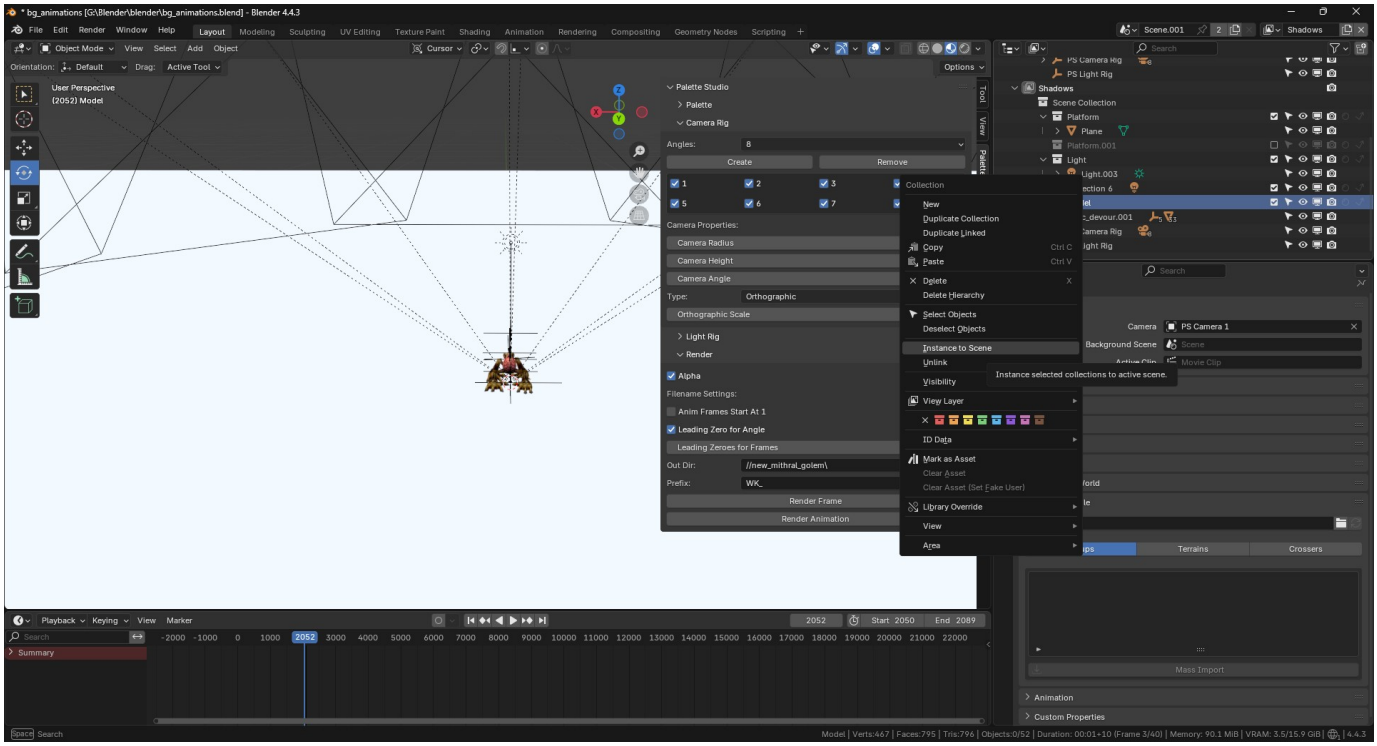


Assuming all the steps have been followed correctly, our Intellect Devourer model should be imported without problem, along with its texture if you saved it in the same directory as the model:

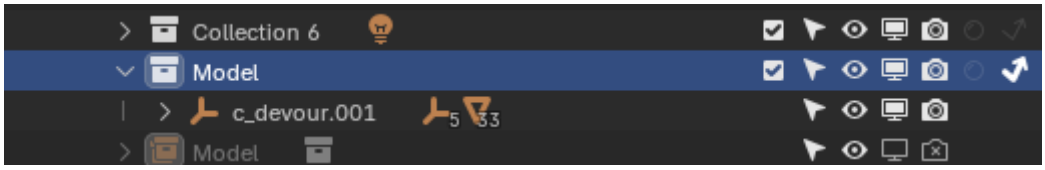


Let’s save this .blend file under its own name. In this case, intellect_devourer.blend.

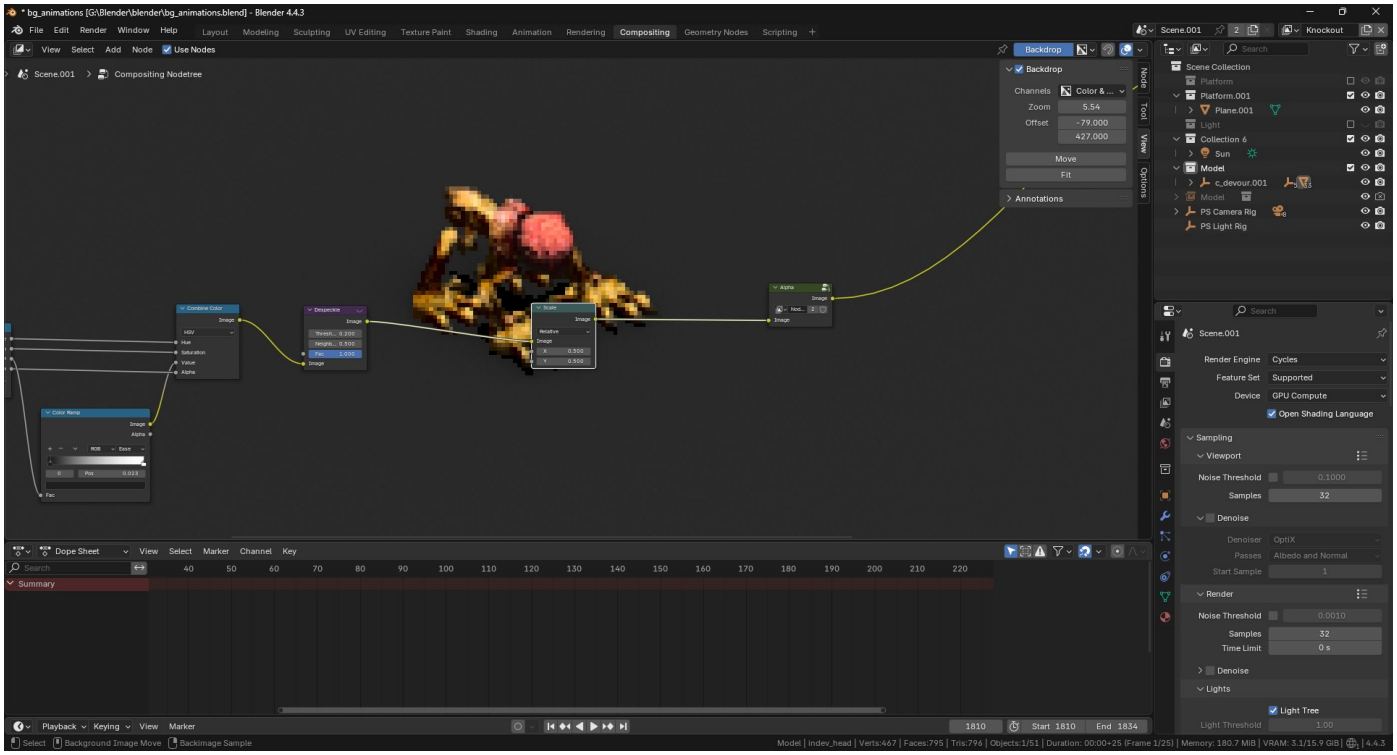
Now while you may want to get to playing with the animations right away, there is one thing we need to do first. With the newer .blend file, select the ‘Shadows’ layer in the top right, right click on the ‘Model’ selection and choose the ‘Instance to Scene’ option.



Then, disable the new instance in Viewports and Renders, and set the Model to Indirect Only by selecting the at the rightmost of the collection. Done correctly, it should look like this.

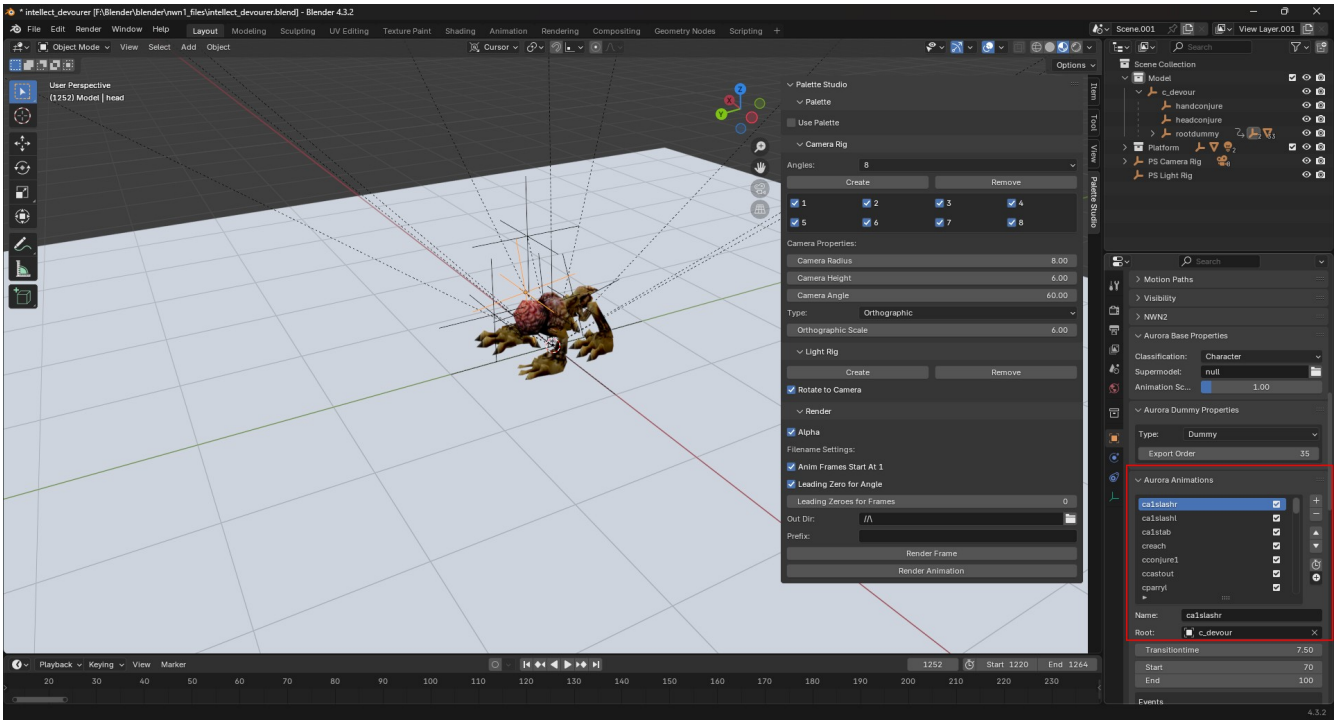


The reason we want to do this is so that we may render the object and shadow separately. As for why we want to do that, this is so that we can ensure that the shadow of the model always uses the correct color (000000, pure black) while we can adjust the object so that it is rendered without any pure black colored pixels. This is to avoid the model itself from ending up with ugly holes in-game, as any black pixels on the model will be treated as part of the shadow and thus be rendered as translucent.



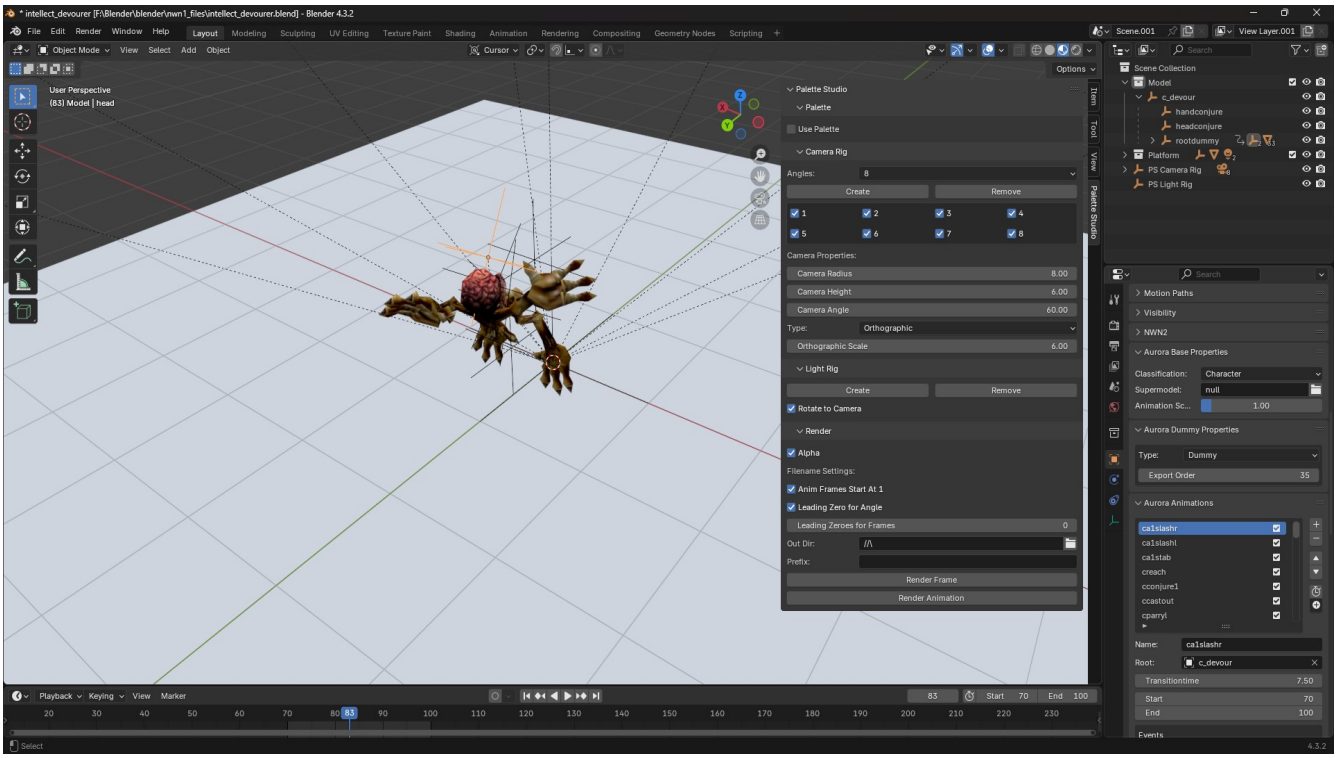
If you want, you can also mess around with the model in Compositing if you like. If you're not familiar with Blender, most of this may look like gibberish, but essentially it's a bunch of post-processing nodes to make the image suitable for converting into an indexed sprite. You can do all sorts of things like changing the color of the model in post-processing, scaling the image, reducing noise, and so on if you know how, but if you don't know what you're doing, I'd advise not touching these.

With that done, we can get to playing with the animations. Under the object ( ) tab in the right, we want to scroll down to Aurora Animations. Here, we can see all the designated animation sequences of the NWN model.



Do note that depending on the model you're using, some animations may not actually be attached to the model itself but to a 'supermodel' instead, which you will also need to find (the file name is as stated in the 'Supermodel' section) that .mdl file via NWN Explorer, export it to the same directory as the .mdl file, and then re-import the .mdl file that you want to render.

Fortunately, in this case the Intellect Devorer model already has its animations, in which case the 'Supermodel' section simply displays as 'null', meaning we don't have to worry about that.



Select any of the animations listed on the right, click the little stopwatch button, and hit

spacebar to play the animation for a preview of what you're rendering. In this case, we've chosen the ca1slashr animation, which is one of the designated attack animations. The NWN animation set will share many of the IE sequences that we need, such as attacking, conjuring, casting, getting hit, dying… etc, etc. In other words, you can simply preview the animations and render the most appropriate ones. In this case, we'll be using the following sequences, which is generally a good starting point for any NWN1 animation:

ca1slashr → A1 (slash)

ca1slashl →A2 (backslash)

ccastout → CA (cast)

ckdbck + ckdbckdie* → DE/SL (die/sleep)

cdamagel → GH

cguptokdb + cgustandb → GU

creadyl → SC (ready)

cpause1 → SD (idle)

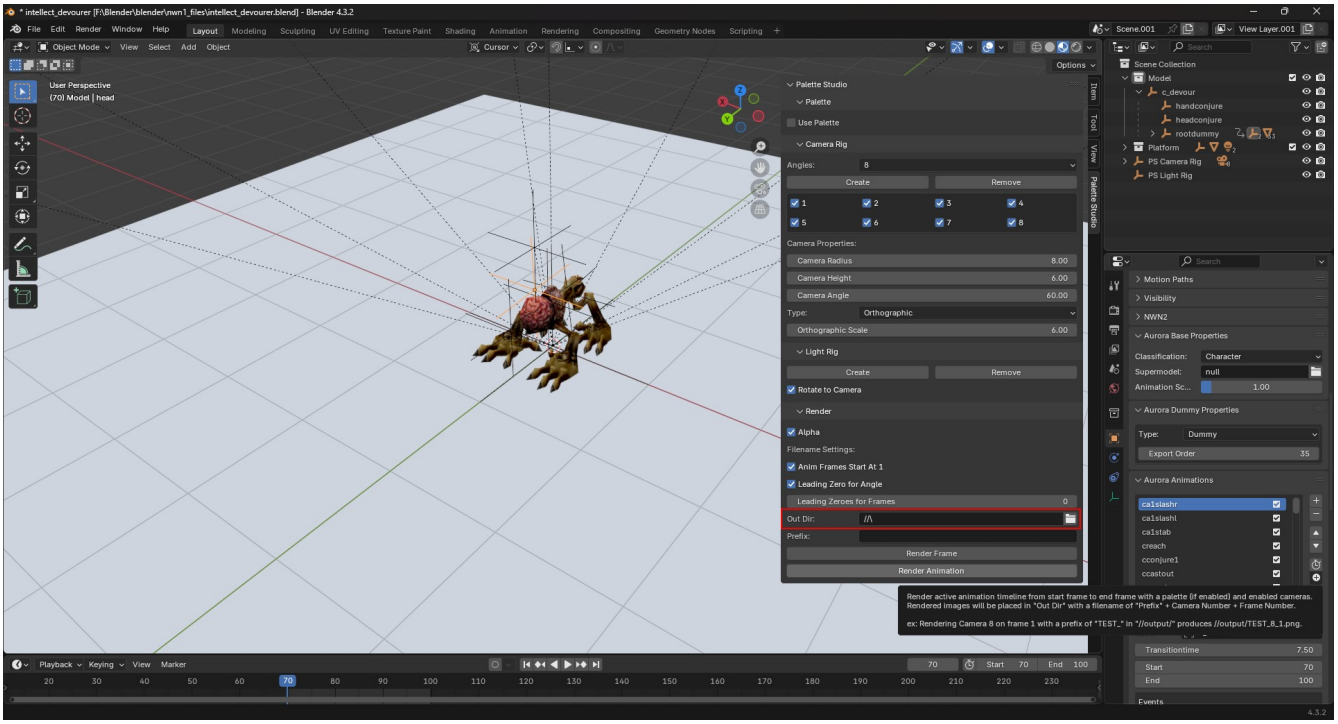
cconjure1 → SP (conjure)

crun → WK (walk)

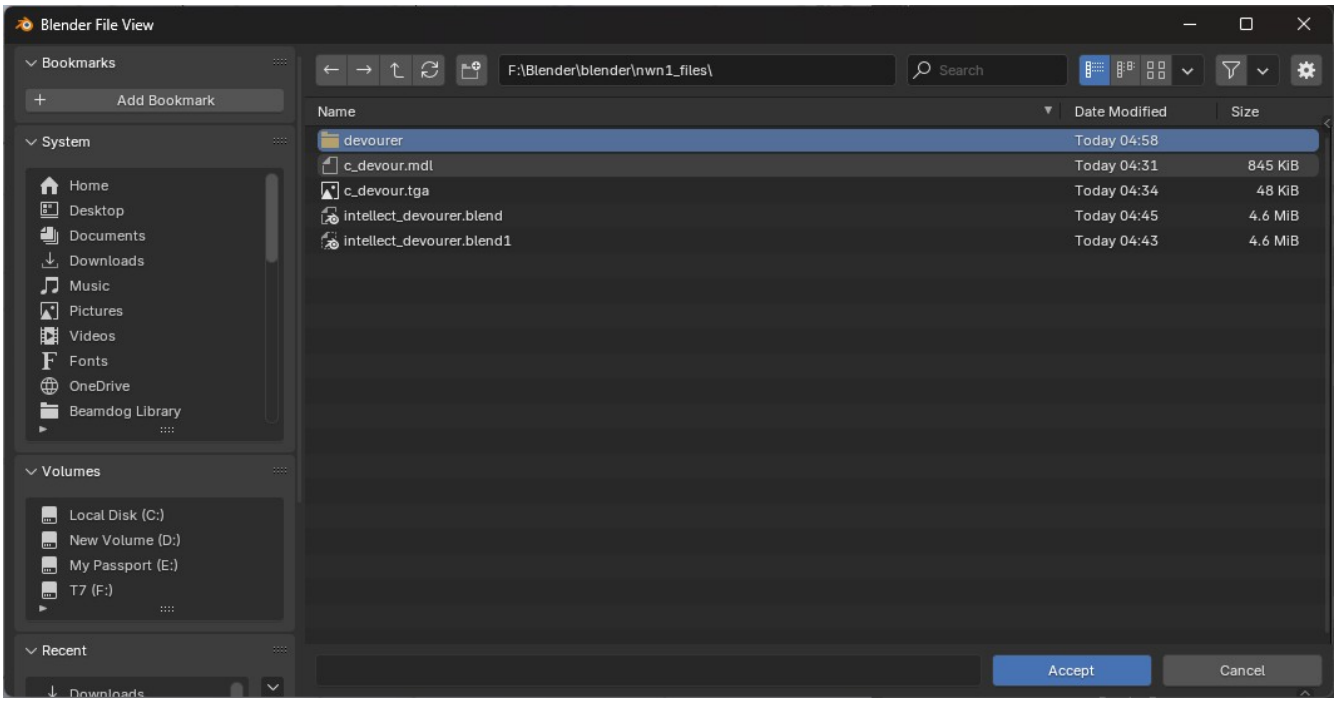
* Since in NWN creatures have a set of animations for being knocked prone, their 'falling over' and 'dying' are split into separate animations. As such, we will be using two sets and combining them for a fluid death animation.

Note that these are not set in stone. If you feel the 'cwalk' animation, for example, for a slower moving creature is more appropriate, then use it instead of 'crun'. In any case, since we now know which sequences we will be using, it's time to render.

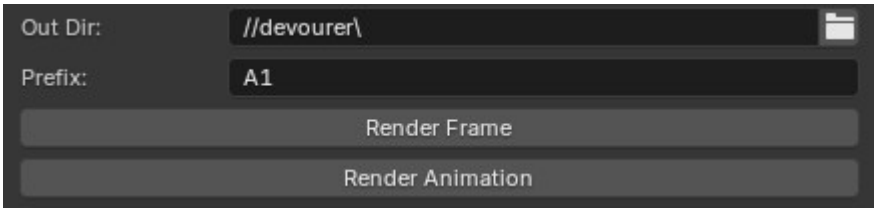
The Palette Studio addon automates the rendering of the eight angles we need for each animation, which makes things a lot more convenient for us. In the dropdown menu, we want to select an output directory for our rendered frames.



We'll be making a new folder here, which I'll be naming 'devourer'.



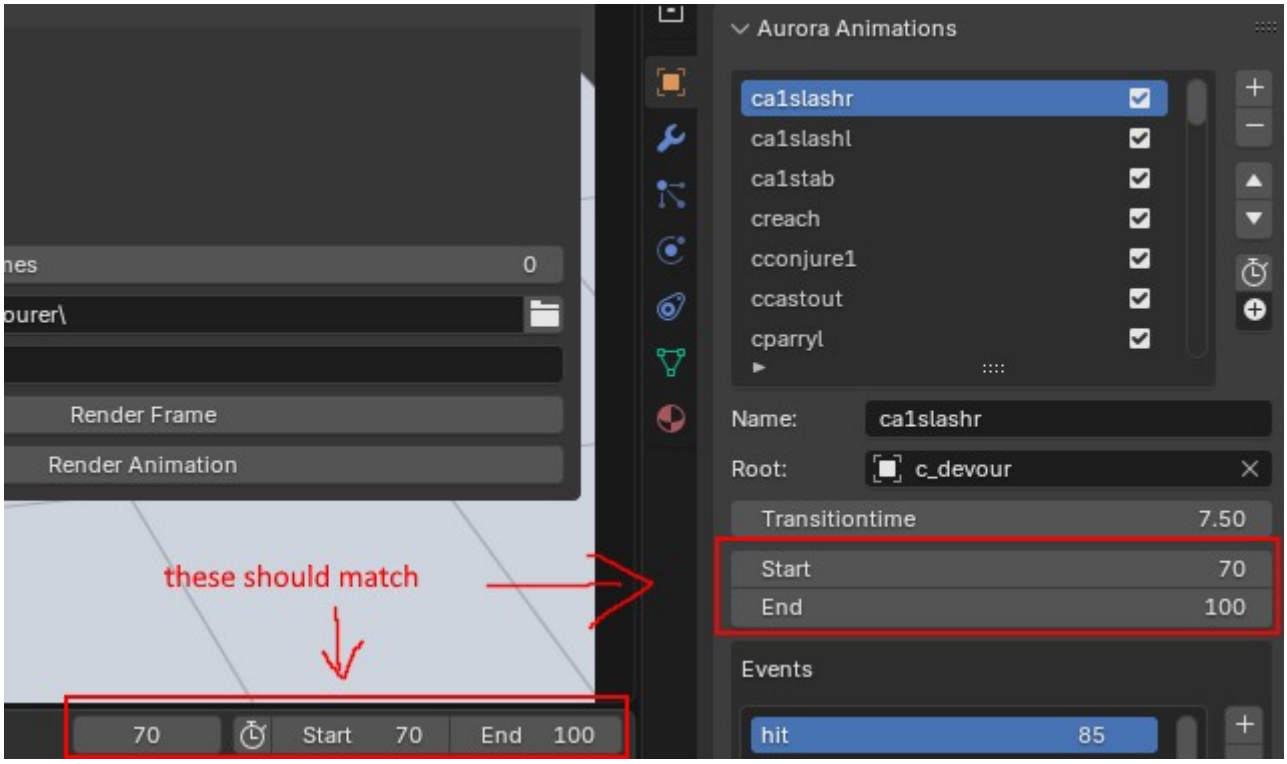
Choose our newly-made folder as our directory. Next, we need to assign a prefix which will be assigned to the names of our rendered frames. For convenience's sake, we'll make the prefix the same as the IE animation sequence we're rendering for. In this case, since we're rendering the 'ca1slashr' animation, it'll be 'A1'.



NOTE: It's up to preference, but I strongly recommend unchecking the Anim Frames Start At 1 option on Palette Studios, which numbers our rendered frames starting from 1 instead of the actual frame we're animating from (for 'ca1slashr', it'll be 70). It makes it a bit harder to read, but this way if you render a different animation and forget to change the prefix, it won't overwrite the files you've already rendered since the output files will have a different starting frame number. Or in the event that Blender crashes mid-render, you can simply look at your rendered images to see what the last rendered frame is numbered, set the current and start frame to number, and continue where you left off.

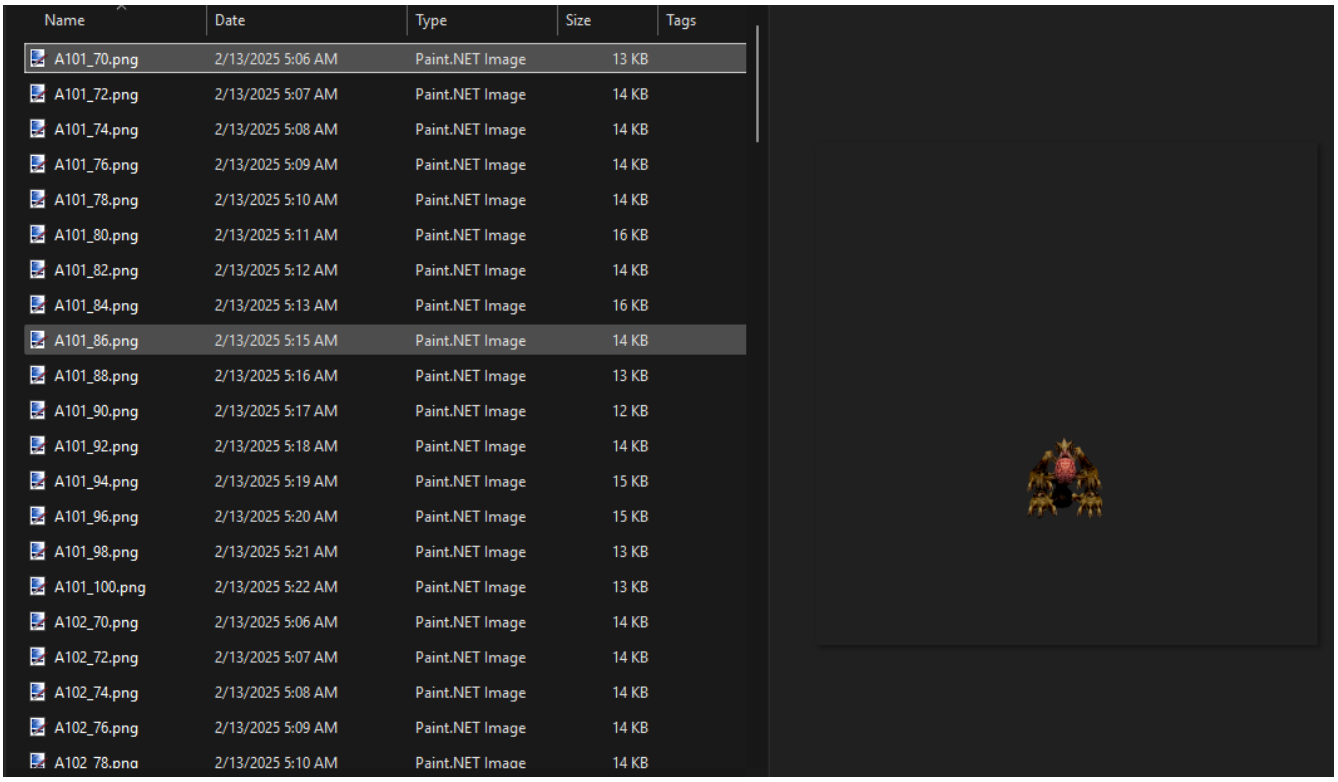
I have provided some default settings for the render, but depending on certain things such as the size of the model you're rendering, you may want to adjust some settings such as resolution, camera height, orthographic scale, and so on to suit your needs. Play around with the camera settings and use Render Frame to produce some still frames to give you an idea of what your eventual animation will look like.

Anyways, we've got everything set up, so now it's time to start rendering. MAKE SURE THE CURRENT AND STARTING FRAMES MATCH AND ARE THE VERY FIRST FRAME OF THE ANIMATION. IF NOT, THE RENDER WILL END UP SKIPPING FRAMES. If unsure, just reset the animation by clicking the stopwatch icon on Aurora Animations.



Simply click the Render Animation button and then wait. Depending on your Render settings (by default, the settings are low, but you may want to sacrifice time for higher quality) and the strength of your hardware, this will take some time. In my case, rendering the frames for this animation took about 2-3 minutes in total.

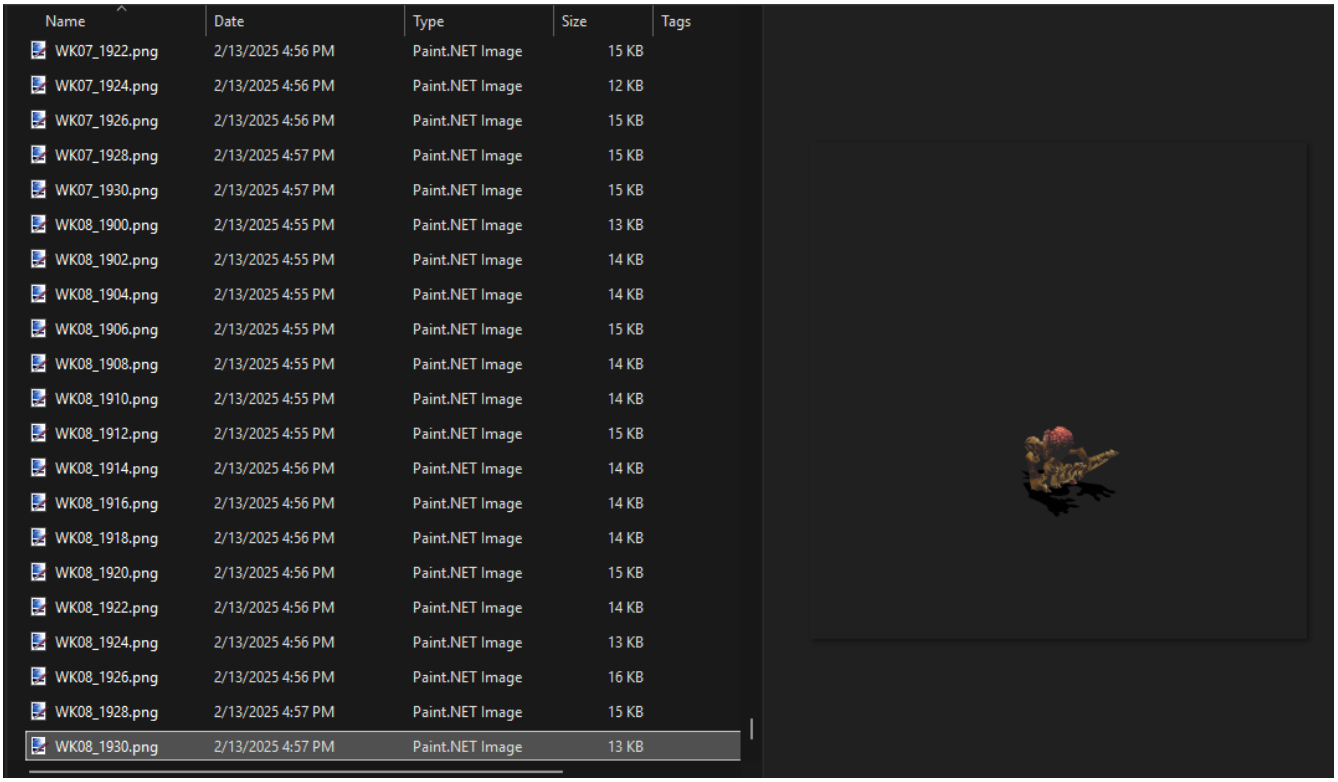
Once Blender is done rendering, go to our new directory to take a look at what we have:



Name	Date	Type	Size	Tags
A101_70.png	2/13/2025 5:06 AM	Paint.NET Image	13 KB	
A101_72.png	2/13/2025 5:07 AM	Paint.NET Image	14 KB	
A101_74.png	2/13/2025 5:08 AM	Paint.NET Image	14 KB	
A101_76.png	2/13/2025 5:09 AM	Paint.NET Image	14 KB	
A101_78.png	2/13/2025 5:10 AM	Paint.NET Image	14 KB	
A101_80.png	2/13/2025 5:11 AM	Paint.NET Image	16 KB	
A101_82.png	2/13/2025 5:12 AM	Paint.NET Image	14 KB	
A101_84.png	2/13/2025 5:13 AM	Paint.NET Image	16 KB	
A101_86.png	2/13/2025 5:15 AM	Paint.NET Image	14 KB	
A101_88.png	2/13/2025 5:16 AM	Paint.NET Image	13 KB	
A101_90.png	2/13/2025 5:17 AM	Paint.NET Image	12 KB	
A101_92.png	2/13/2025 5:18 AM	Paint.NET Image	14 KB	
A101_94.png	2/13/2025 5:19 AM	Paint.NET Image	15 KB	
A101_96.png	2/13/2025 5:20 AM	Paint.NET Image	15 KB	
A101_98.png	2/13/2025 5:21 AM	Paint.NET Image	13 KB	
A101_100.png	2/13/2025 5:22 AM	Paint.NET Image	13 KB	
A102_70.png	2/13/2025 5:06 AM	Paint.NET Image	14 KB	
A102_72.png	2/13/2025 5:07 AM	Paint.NET Image	14 KB	
A102_74.png	2/13/2025 5:08 AM	Paint.NET Image	14 KB	
A102_76.png	2/13/2025 5:09 AM	Paint.NET Image	14 KB	
A102_78.png	2/13/2025 5:10 AM	Paint.NET Image	14 KB	

As we can see, we now have a bunch of .png files, composing each frame of our animation. These are assorted by prefix, camera, and frame, so A101_70 would be the first frame of the animation rendered from the perspective of the first camera angle. You may notice that these files are all even-numbered, meaning we've skipped all the odd-number frames, and there's a reason for this. The IE games are designed for 30 fps, and the speed of the game is tied to the framerate. This means that animations drawn to run at 60 fps will feel particularly sluggish in the IE game engine. It's up to you whether you want to do this (if not, just go to the Output () tab and set the Step setting in Frame Range from 2 to 1), but I find this preferable both because it reduces the render time and makes the animations fit the speed of the game better.

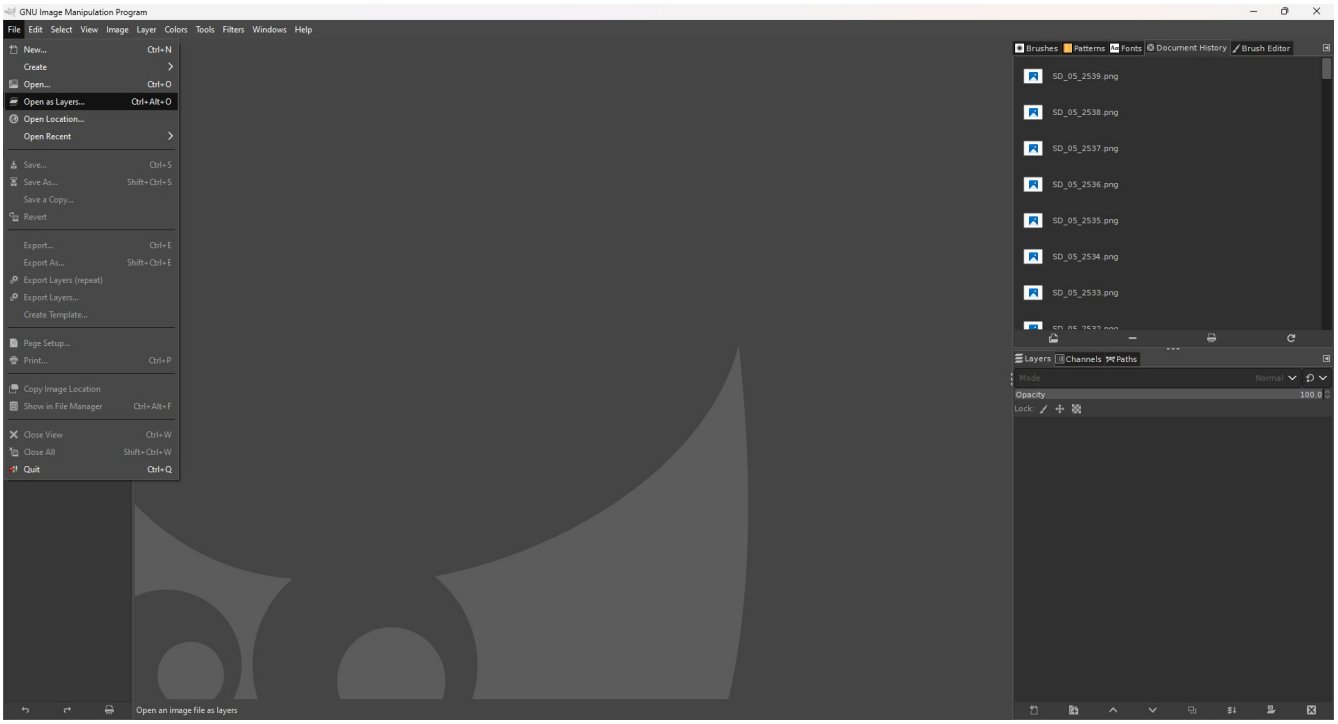
In any case, we now have to do this for every animation sequence that we need. So we set the animation to ca1slashl, change the prefix to A2, click Render Animation, wait, and so on…



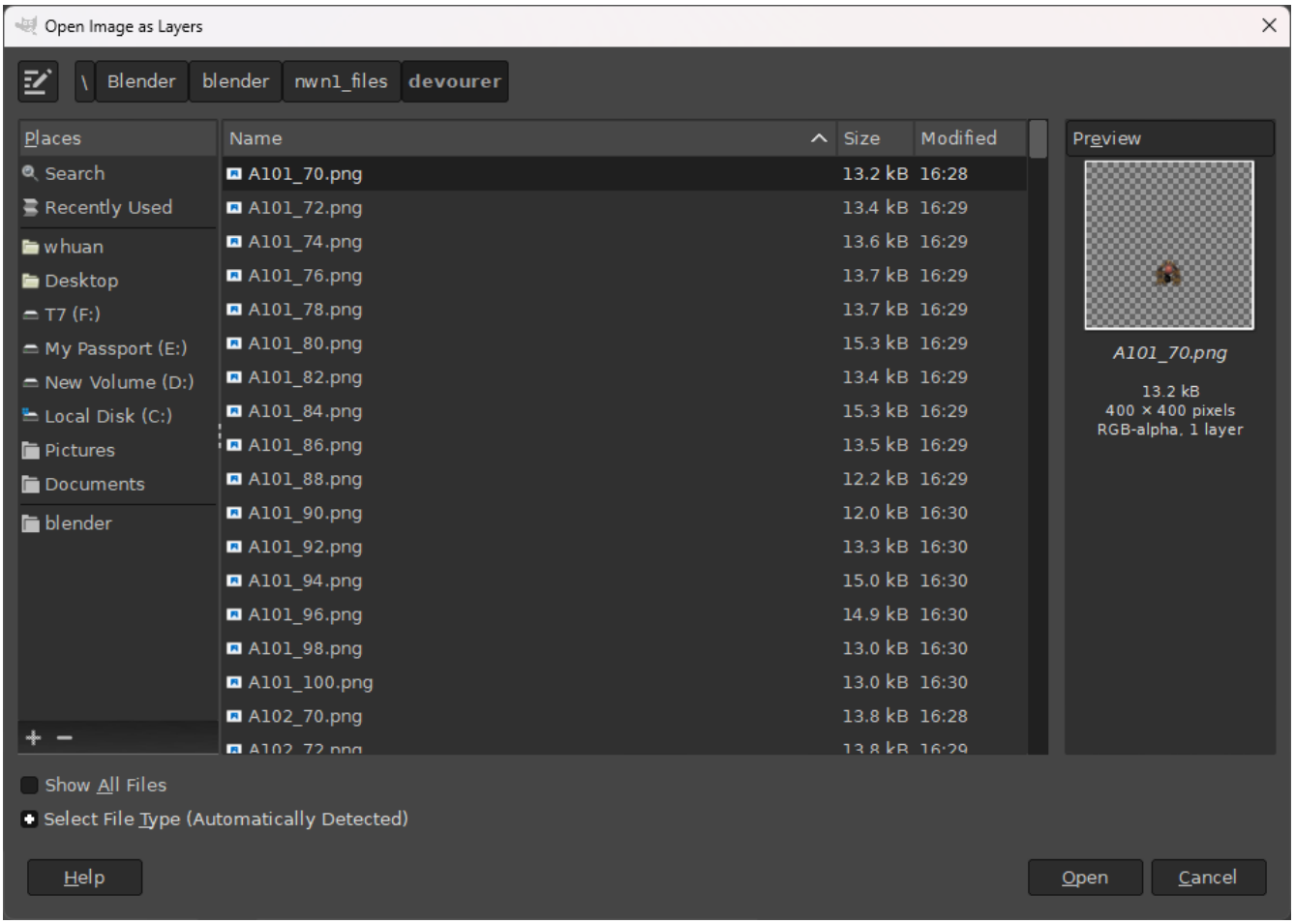
Once we're done, we should have a collection of ~1500 .png images (number varies depending on what you've rendered). Congratulations, we're done with Blender. But we can't just use these frames as they are. As mentioned before, the Infinity Engine uses indexed images, which these are not. If we added these into .bam files as they are, the resulting animation is not going to look right. Instead, we now need to run these through an image editing software, in this case, GIMP.

Indexing Images

NOTE: Before you open GIMP, make sure you have the Export Layers plugin installed. If you don't know how to install the plugin, simply go to Edit → Preferences → Folders → Plug-ins, open either of the folders listed, and copy the export_layers and export_layers.py files into the plug-ins folder.

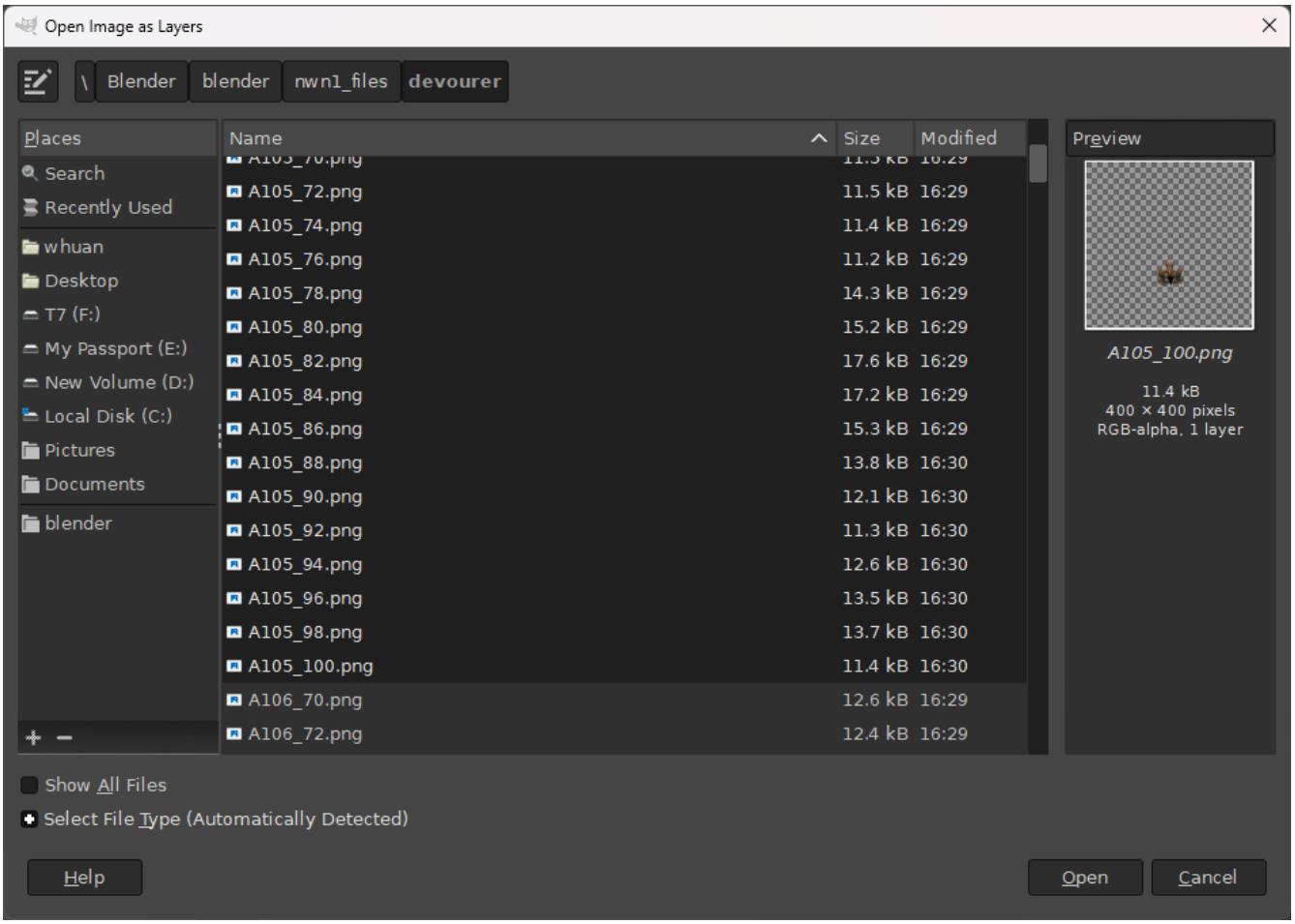


Here comes the tedious part. Go to Files→Open As Layers, and go to the directory in which we rendered all of our animation frames.

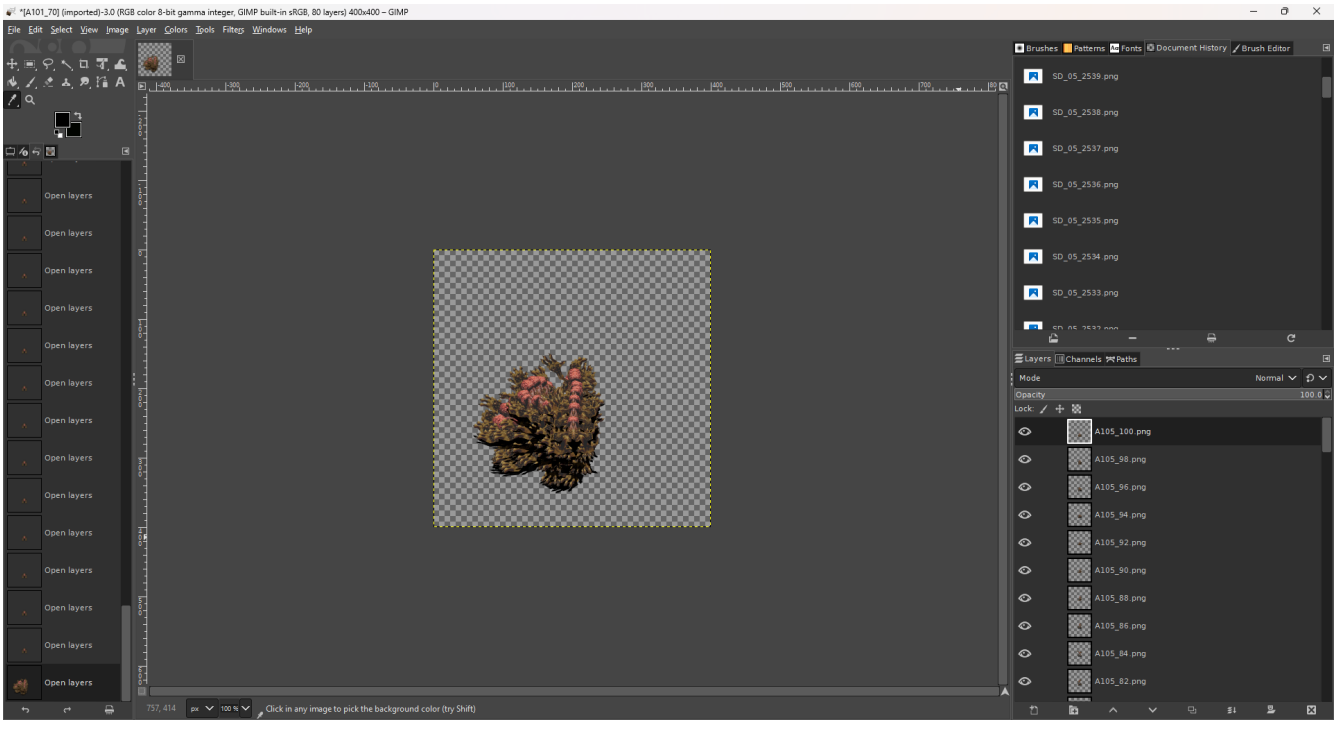


What we want to do now is open all the frames that we want to add to each individual .bam animation file. This may get a little complicated, but the .bam files essentially consist of the frames for each animation sequence in select directions. For the E000 animation type we're

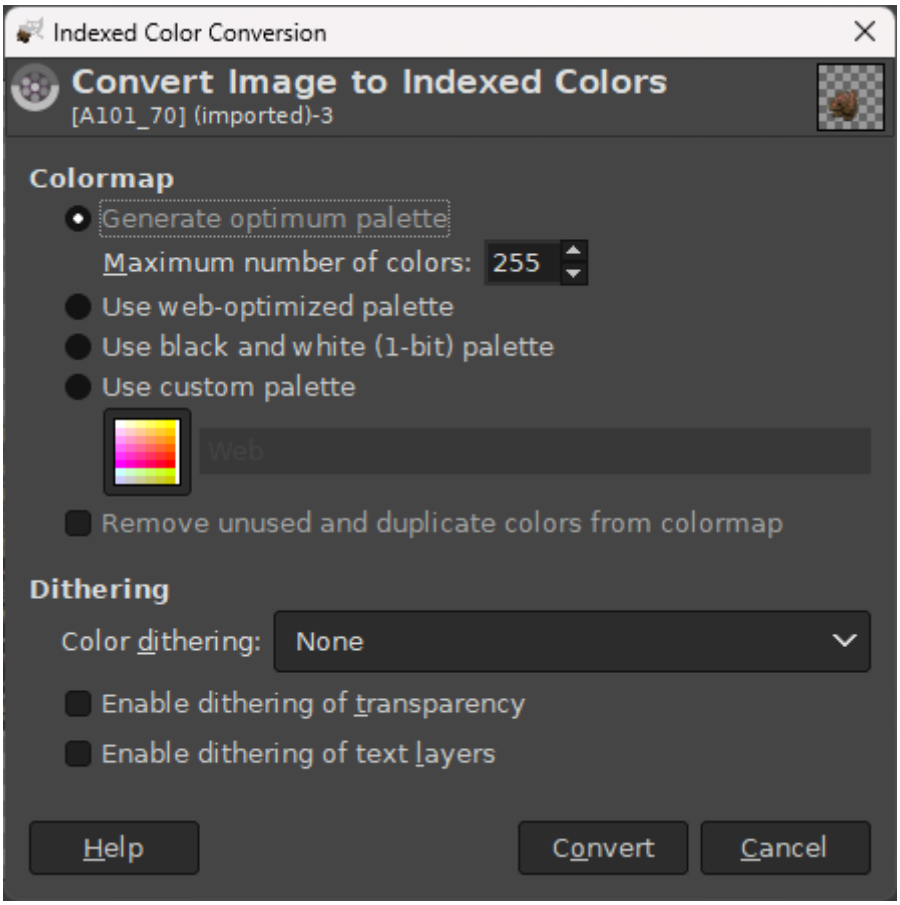
using, the left-facing .bam files consist of the 1-5 range and the ‘E’ right-facing .bam files consist of the 6-8 range. In other words, for the first .bam file we want to make, xxxxA1.bam, we’ll be opening all of the A101-A105 images.



Loading each individual image as its own layer will take some time. Once all the images have loaded, you should have something that looks like this.

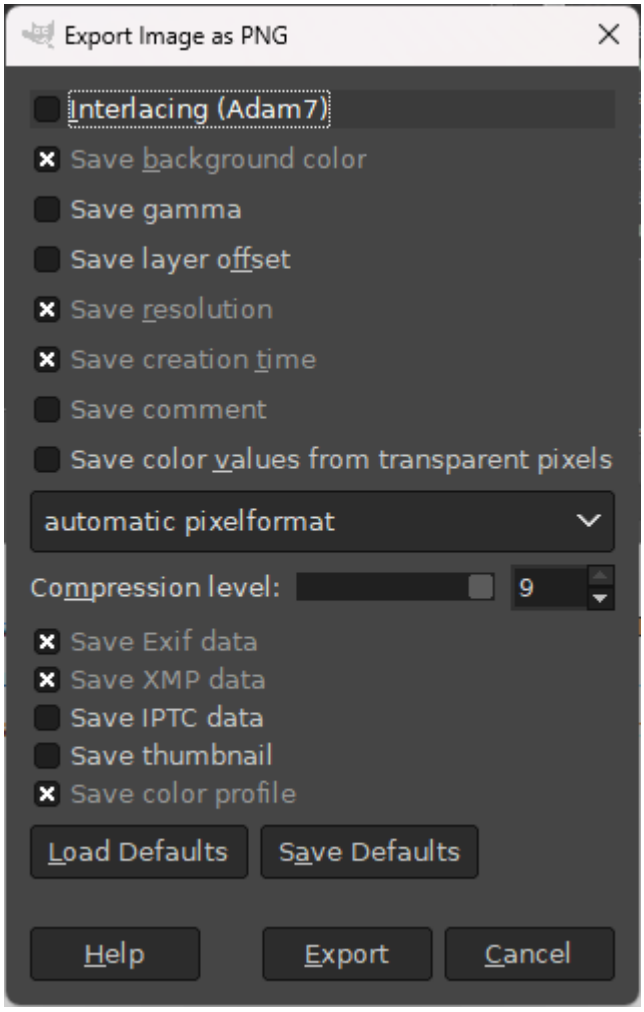


Now we want to go to Image → Mode and select Indexed.



I recommend using these settings. Now we simply click Convert and we have ourselves a bunch of indexed images. What we're doing is making sure all of these images are sharing the same 256-color palette. Now that we've done that, we can export our individual frames using the Export Layers plugin.

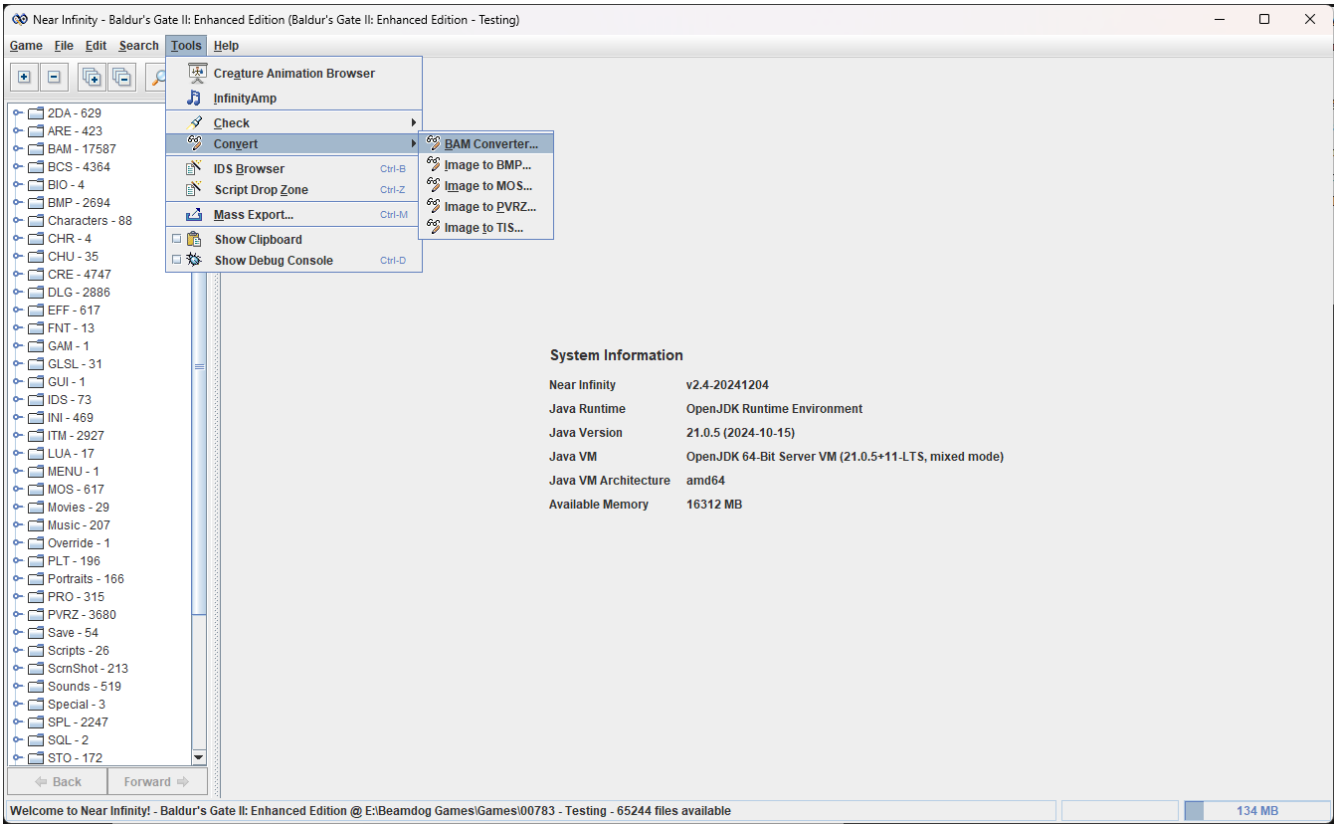
I recommend creating a new folder, calling it 'output' or 'indexed' or something like that, and exporting all the images in there. You don't want to do it in the same folder as your rendered images since they'll overwrite them, and you may still want those in case you realize you messed up later.



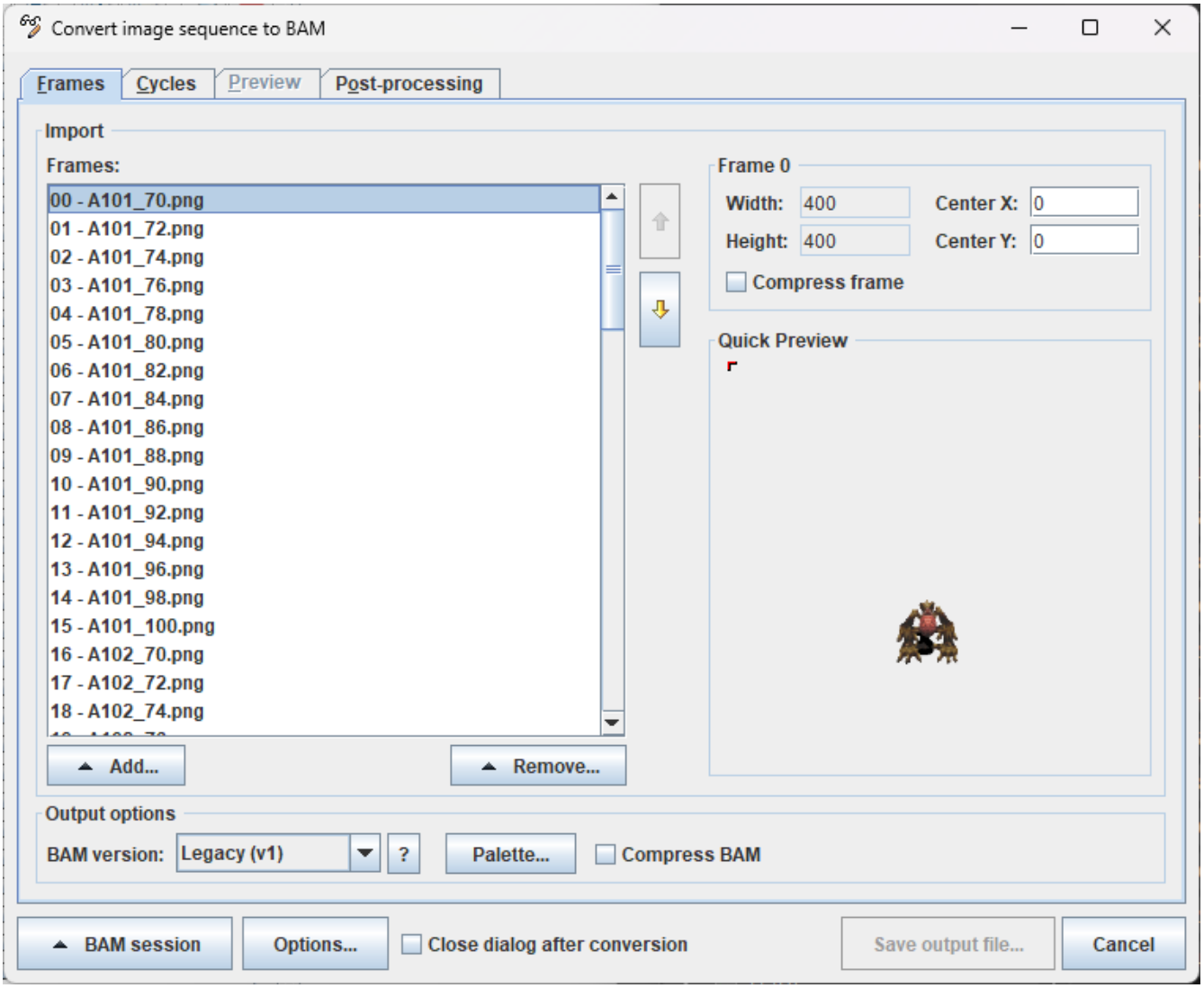
You'll get a prompt before exporting. Use these settings. The exporting, again, will take some time. Once you're done, you'll have a bunch of indexed images in your new folder, ready to be added to .bam files. Now you just have to do close the image and redo this process that for every animation set you're using. For the next file, xxxxA1E.bam, you'll be using the A106-A108 image files, then A201-A205 for xxxxA2.bam, A206-A208 for xxxxA2E.bam, and so on. This is by far the most boring part of the process.

Now technically, you could just load all your images as layers in a single batch and index them all before exporting if you wanted to save time. The problem with doing so is that *all* your frames will be drawing from the same 256-color palette, which means the resulting animation will be of significantly lower quality. I admittedly used to do this myself, but separating the images by their assigned .bam files will ultimately result in a better-looking animation. It's up to you whether you want to save time or not, and for smaller models you may be able to get away with it. In any case, once you're done with indexing and exporting your frames, it's time to actually make them into .bam files.

Creating .bam Files



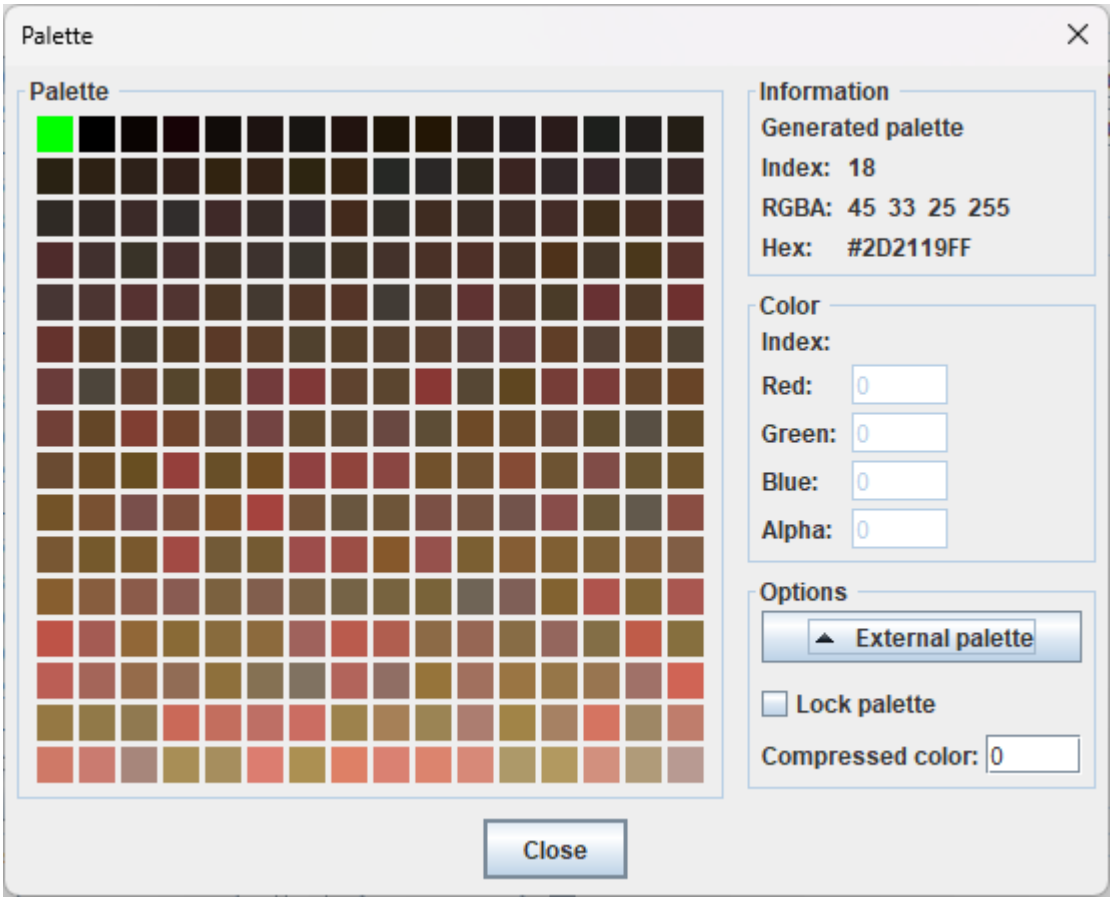
Open NearInfinity, go to Tools → Convert and open the BAM Converter.



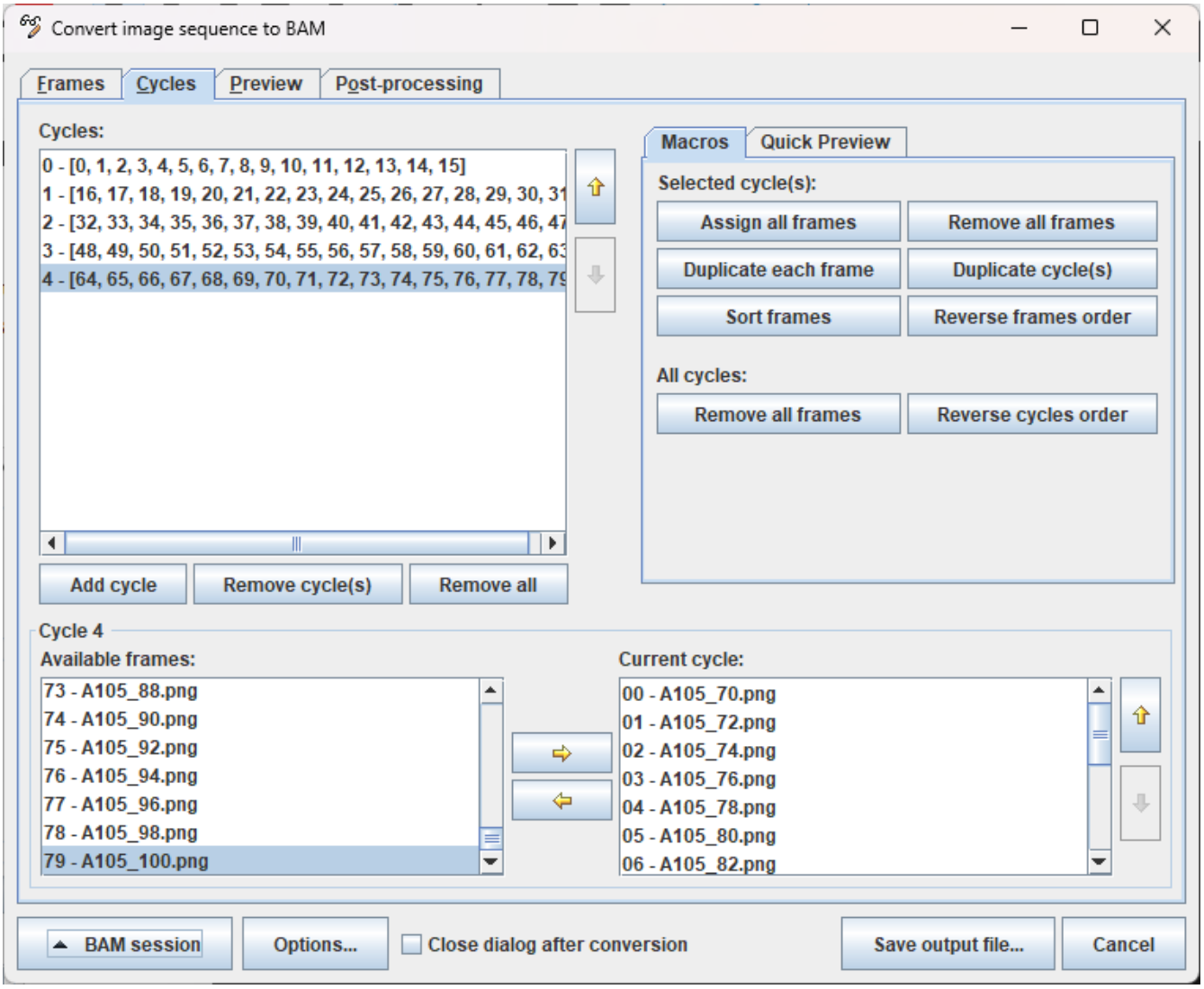
In the BAM Converter, we want to add all the frames we want to use for our first .bam file,

A1. If you have a recent version of NearInfinity, you can simply drag and drop the files you want from your ‘output’ folder into the BAM Converter (the resulting images may not be in the right order, so you may have to rearrange things using the arrow buttons. In my experience, dragging from the first file you want to add, A101_70 in this case, usually prevents this)

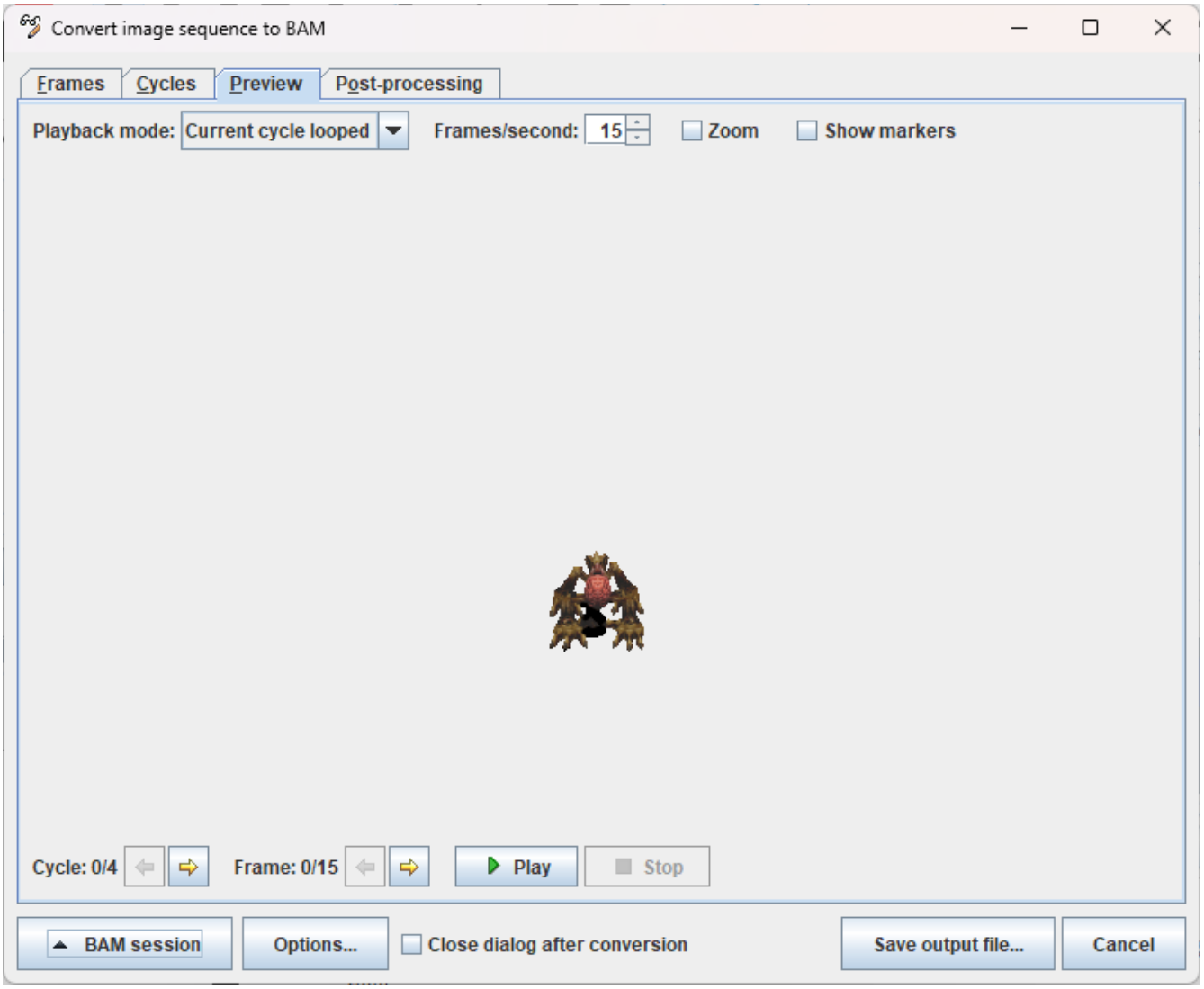
Firstly, we want to make sure there’s no problems with the animation’s palette. Assuming you’ve done the indexing through GIMP correctly, your palette should look like this, with all your colors arranged in a nice orderly manner.



So now we know the palette’s correct. Time to sort out our Cycles. Go to the Cycles tab and add five cycles from 0-4. Then you want to add the frames for each direction of the animation into their respective cycle. All the A101 files go into 0, A102, into 1, etc…

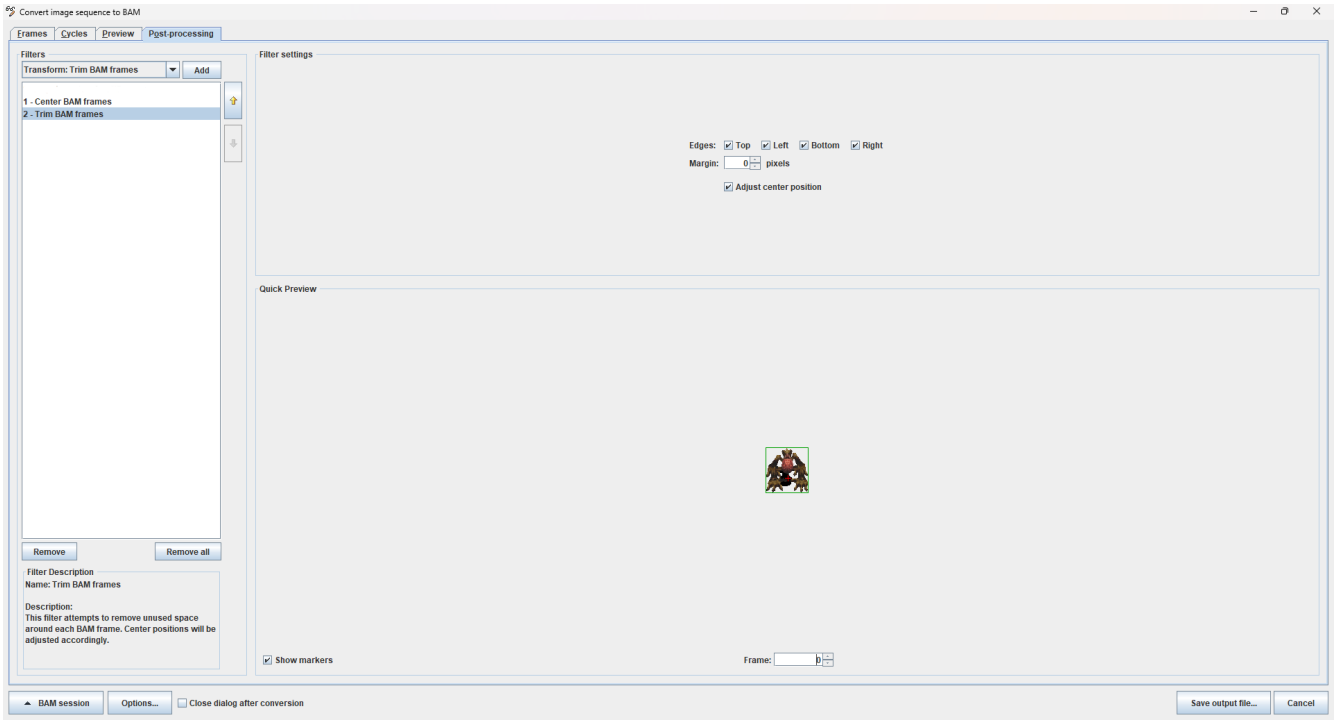


Once you have all your frames sorted, it’s time to go into the preview to see what we have.



Play the animation and switch between the cycles to make sure everything looks right. We're very close to having our first finished animation, but we're not quite done yet. Go to Post-processing.

You'll probably need to set the BAM Converter to fullscreen to fit your animation. In post-processing we can adjust the animation through various filters as we please. Before doing anything, make sure to turn on Show markers. There's a lot we can do in this section, from changing the animation's hue and saturation, making the animation smaller, or whatever else you feel like doing. You can play with the filters as you want, but in this case, I'll only be doing some essential adjustments.

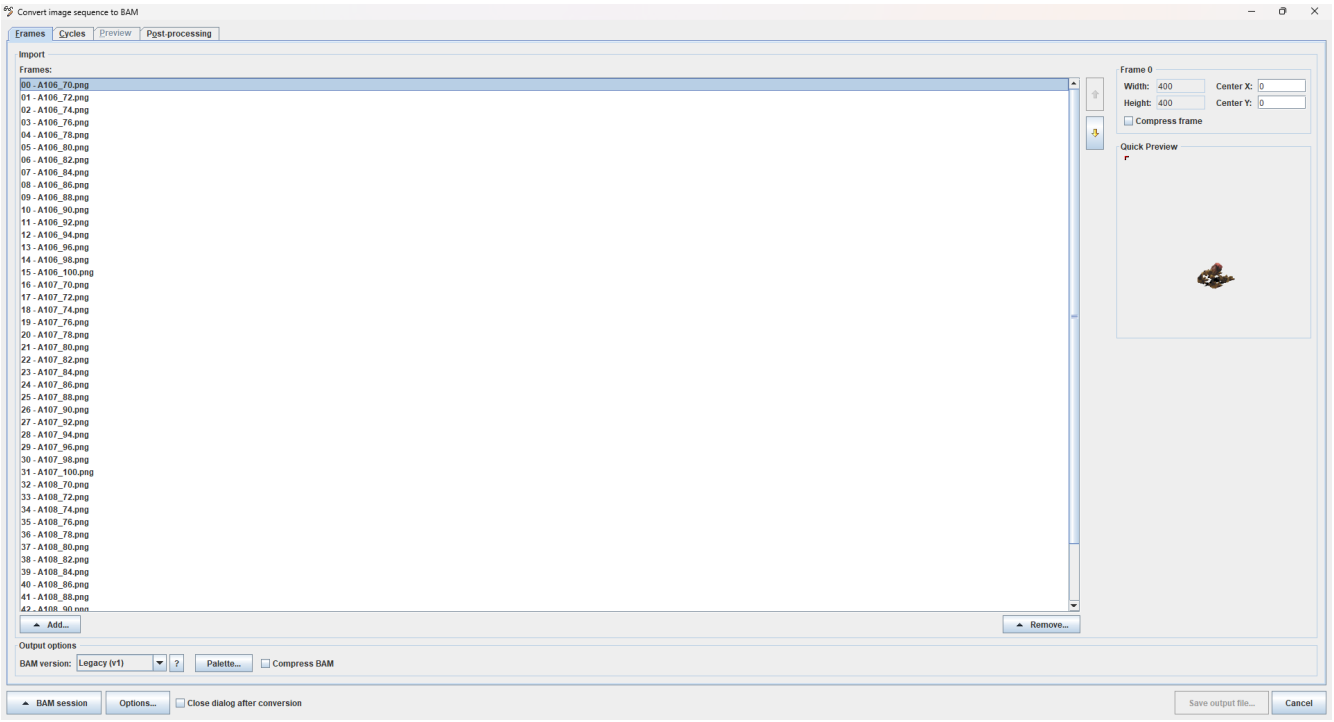


In most cases, the only adjustments you should need to make are Center BAM Frames and Trim BAM Frames. Uncheck the Auto-adjust center position button, and move the Center X and Y co-ordinates until the cross marker is positioned between your model's legs. This determines the position the animation will be played in relative to the creature's green circle in-game. If we just left it at 0,0 where it starts, your animation will be completely off-center from where it should be, and we don't want that. After that, I reduced the size of the model by 70% (you could simply render the frames as smaller in Blender, but I prefer to render larger frames and scale down in post-processing to my liking). After centering and resizing the frames, I then use Trim BAM frames to remove the transparent frames on the edges. Trimming isn't actually essential, but it should cut down on file size and there's not much reason not to do it.

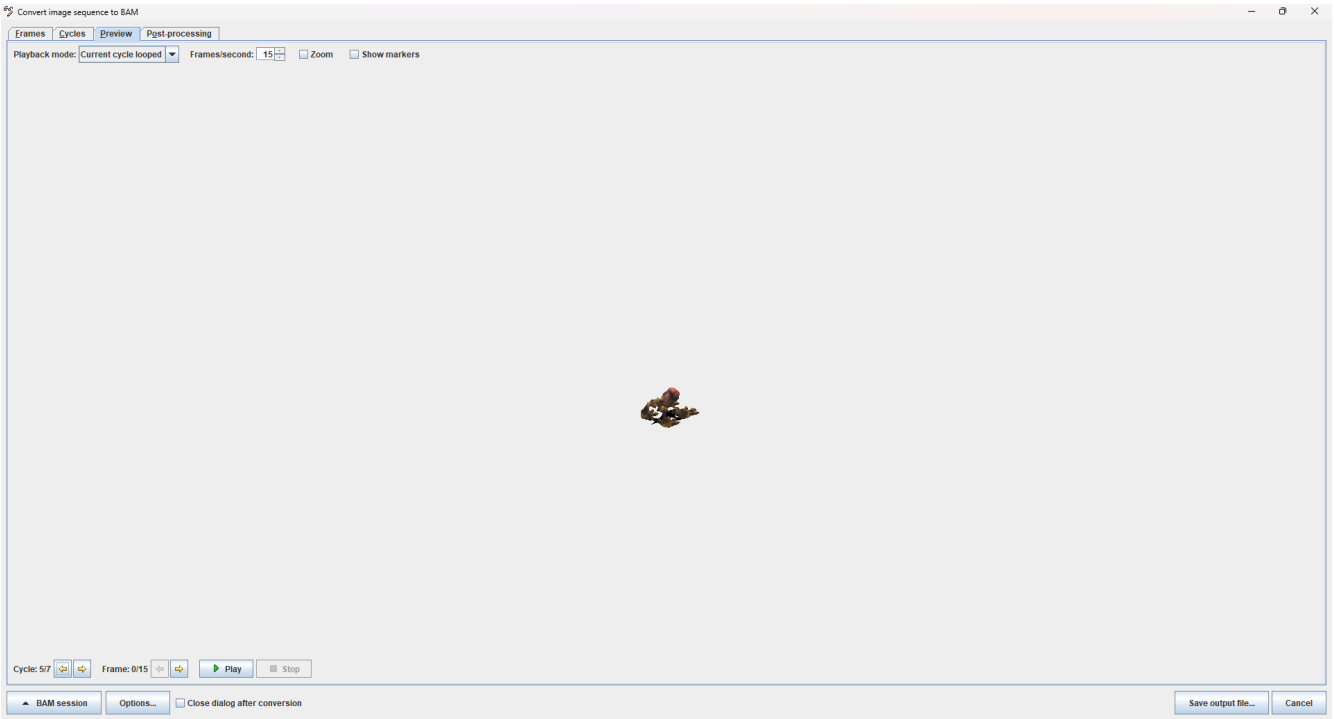
We're done preparing the animation now, but don't save it just yet. Go to BAM session in the bottom left corner and select Export session. Save the .ini file wherever you find convenient and, when prompted, UNCHECK EVERYTHING EXCEPT Filter configurations. This lets us save our post-processing settings so that we can repeat them for the other .bam files without having to remember and redo them manually every time.

Now we can save the .bam file. I'll be creating a new mod folder, named artisan_animations, creating an 'animations' folder in that, and saving it under a new name. We'll need a four-letter reference to draw upon later. Since we're animating an intellect devourer and my modder prefix is C0, I'll be naming it C0IDA1.bam.

With that, we have our first complete .bam file. Repeat the process for the next animation, which is going to be the right-facing version of the A1 animation – C0IDA1E.bam.

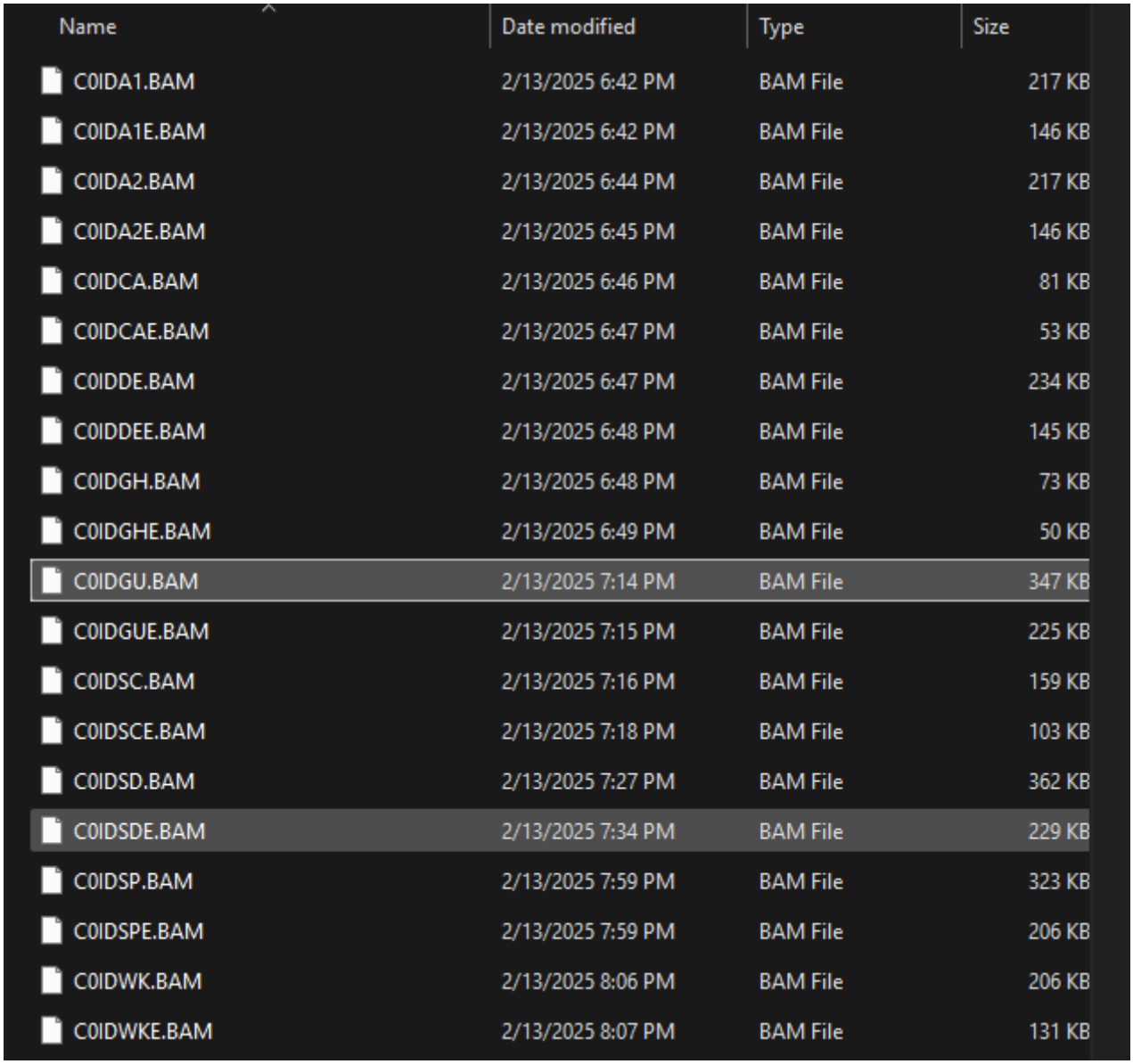


This time, we'll be copying all the A106-A108 image files, and for Cycles, we'll be creating eight cycles (0-7), and assigning the frames to the *last three cycles* (5-7) only. Once you go to preview, you won't see anything initially, since it'll be on the first empty cycle. If you change to the sixth one, however, you'll see the animation facing in one of the right directions.



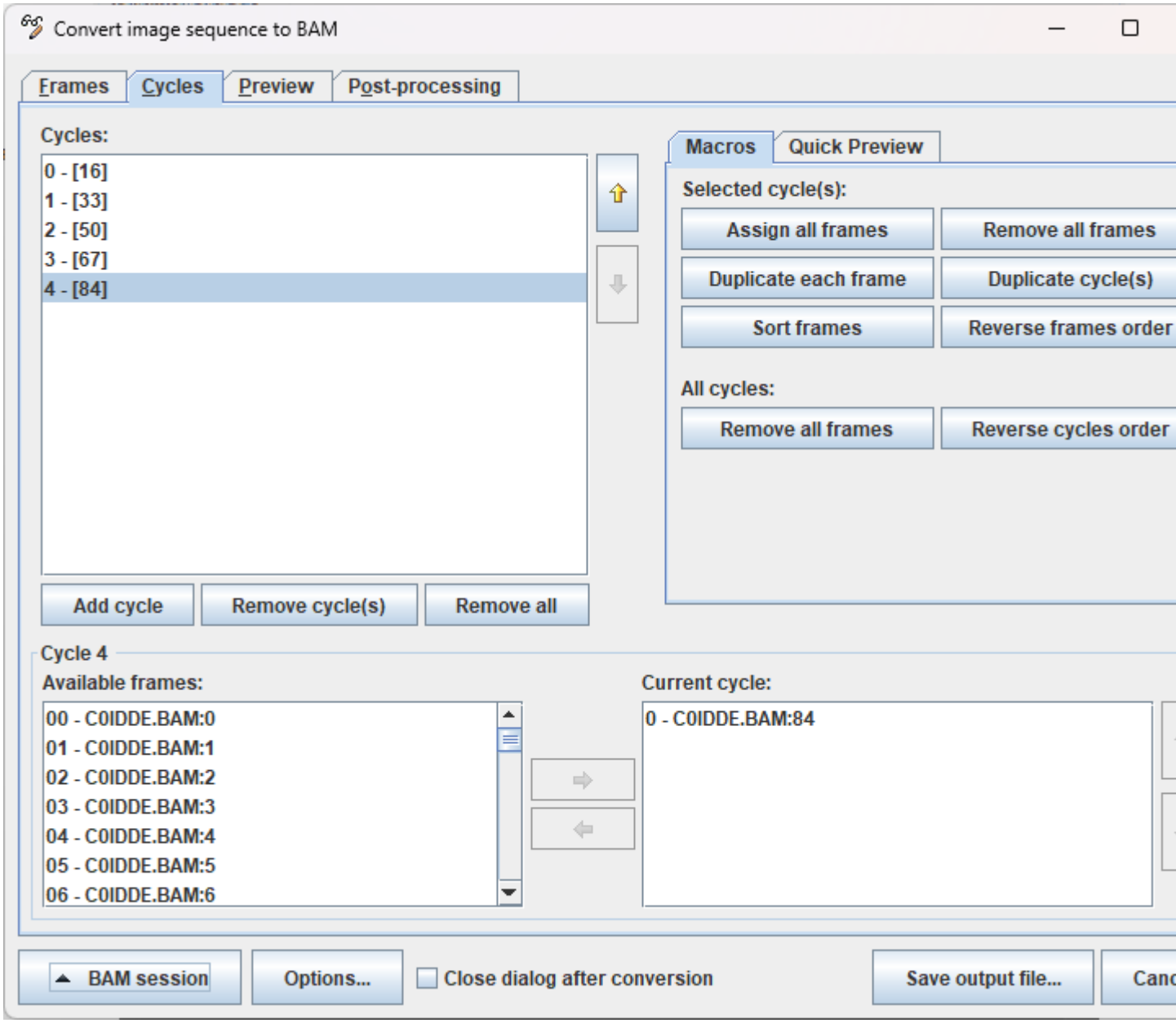
Now, go to BAM session again, select Load recent sessions, and choose the .ini file you saved from the first .bam file. This will copy the post-processing settings we used onto this one, saving us the effort of doing it again.

Save this new .bam file as xxxxA1E.bam. Once you've done this for every animation, you'll have a bunch of .bam files like this.



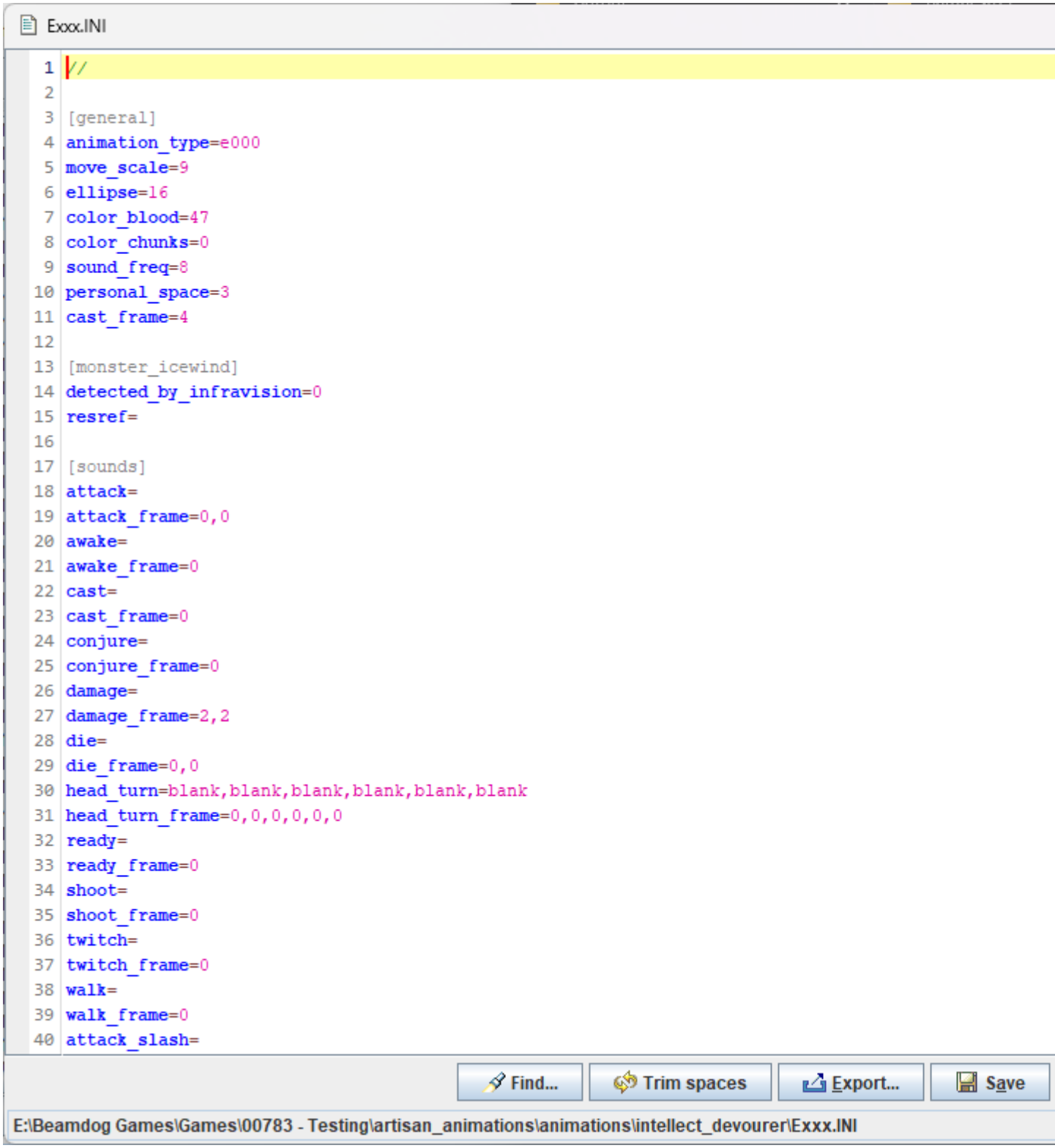
Name	Date modified	Type	Size
C0IDA1.BAM	2/13/2025 6:42 PM	BAM File	217 KB
C0IDA1E.BAM	2/13/2025 6:42 PM	BAM File	146 KB
C0IDA2.BAM	2/13/2025 6:44 PM	BAM File	217 KB
C0IDA2E.BAM	2/13/2025 6:45 PM	BAM File	146 KB
C0IDCA.BAM	2/13/2025 6:46 PM	BAM File	81 KB
C0IDCAE.BAM	2/13/2025 6:47 PM	BAM File	53 KB
C0IDDE.BAM	2/13/2025 6:47 PM	BAM File	234 KB
C0IDDEE.BAM	2/13/2025 6:48 PM	BAM File	145 KB
C0IDGH.BAM	2/13/2025 6:48 PM	BAM File	73 KB
C0IDGHE.BAM	2/13/2025 6:49 PM	BAM File	50 KB
C0IDGU.BAM	2/13/2025 7:14 PM	BAM File	347 KB
C0IDGUE.BAM	2/13/2025 7:15 PM	BAM File	225 KB
C0IDSC.BAM	2/13/2025 7:16 PM	BAM File	159 KB
C0IDSCE.BAM	2/13/2025 7:18 PM	BAM File	103 KB
C0IDSD.BAM	2/13/2025 7:27 PM	BAM File	362 KB
C0IDSDE.BAM	2/13/2025 7:34 PM	BAM File	229 KB
C0IDSP.BAM	2/13/2025 7:59 PM	BAM File	323 KB
C0IDSPE.BAM	2/13/2025 7:59 PM	BAM File	206 KB
C0IDWK.BAM	2/13/2025 8:06 PM	BAM File	206 KB
C0IDWKE.BAM	2/13/2025 8:07 PM	BAM File	131 KB

And that’s it, we have our animation files. Well, almost. We’re still lacking the SL (sleep/unconscious) and TW (dead/twitching) animations, but we can simply copy those based off our DE (dead) animation. Copy xxxxDE.bam and xxxxDEE.bam and rename them to xxxxSL.bam and xxxxSLE.bam respectively, and we have our SL sequences. As for TW, we’ll need to get back into BAM Converter for that. Drag the DE.bam file into NearInfinity and click Edit BAM, which will open up the .bam file in BAM Converter. We want to go into Cycles and delete *every* frame from each cycle *except* for the last one.

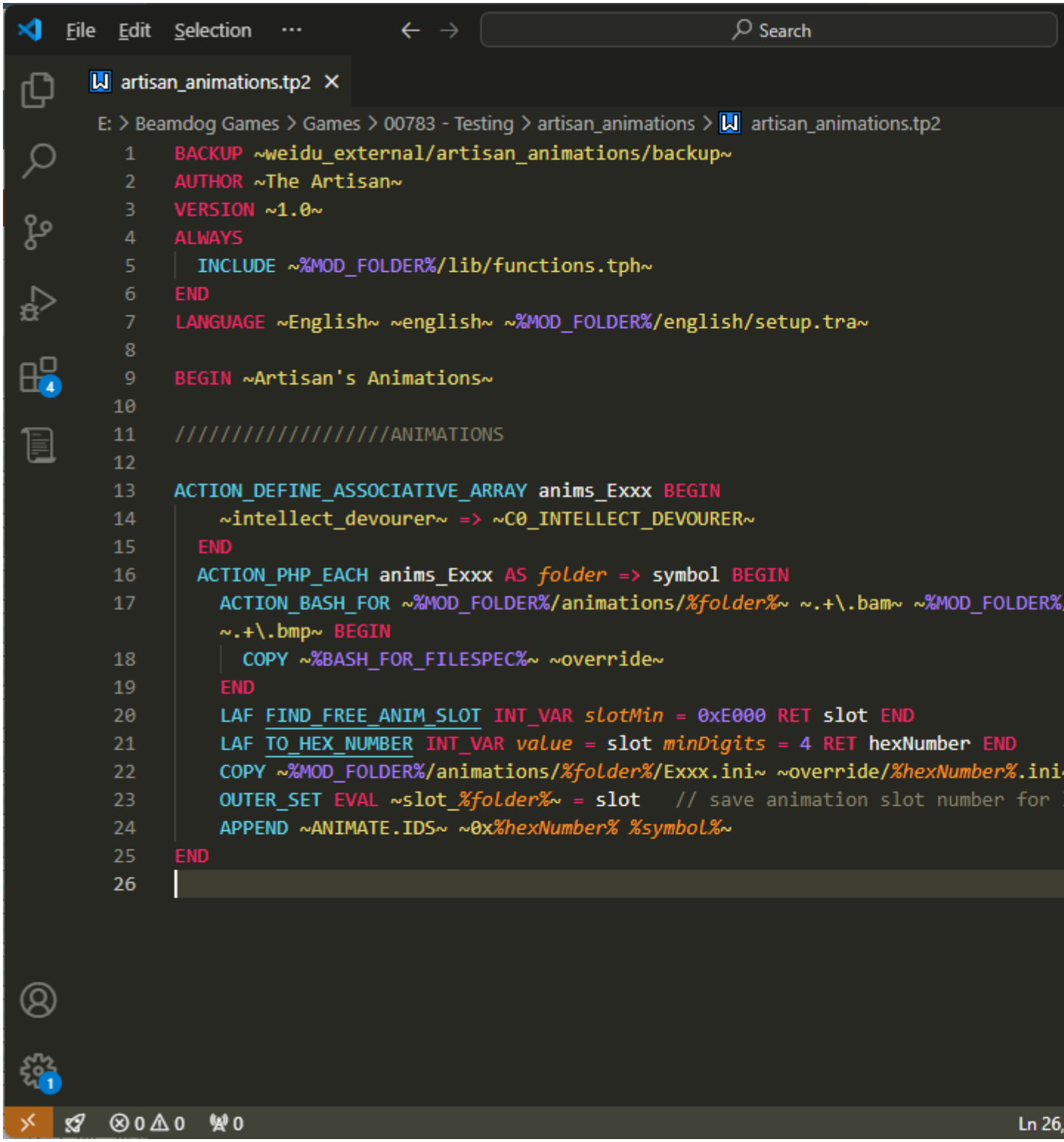


Next, go back to Frames and click Remove → Drop unused frames. Again, this isn't completely necessary, but the additional frames from the DE sequence are not being used anyway and doing so cuts down on the file size. Save the resulting file as xxxxTW.bam, and repeat the process for the DEE.bam file, renaming it to xxxxTWE.bam.

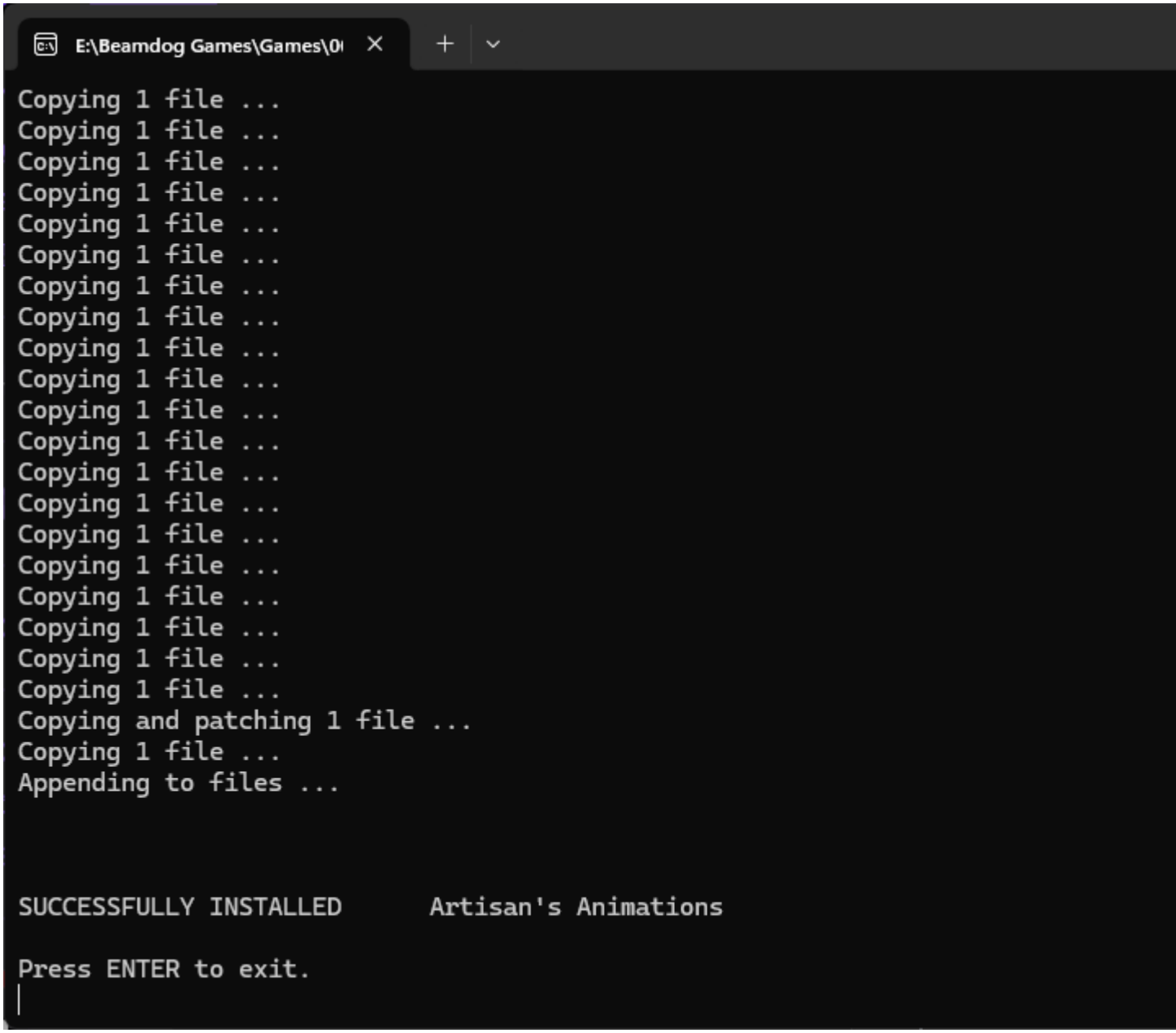
And after all that work and wait, we have all the animations we need. Now we just need to write an .ini animation file. Using the Exxx.ini file in my mod folder as a reference, we can adjust certain properties of the creature, such as their base movement speed, the radius of their green circle, their creature size for determining pathing, and so on.

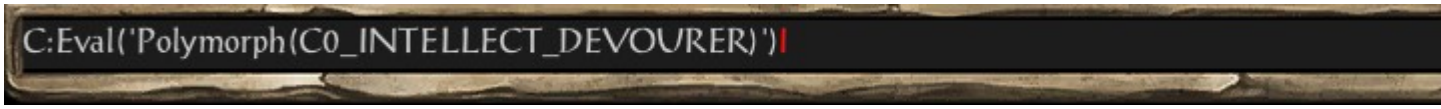


You can also attach sounds to specific animations if you want, but in this case, the only vital change I'll be making is to add to the resref. Since my files are prefixed with C0ID, I'll be writing 'resref=C0ID'. With that, all the resources are ready, and it's time to install the animation via WeiDU.



Using the help of macros, we can append to the ANIMATE.IDS using the first free animation slot that WeiDU can find to ensure compatibility.





And there you have it. One ugly little brain-dog, fully animated and playable.

As for any closing thoughts… those who have followed this guide through to the end will probably think this whole process has been extremely long and slow, and to put it simply… it is. The IE games don't get much attention in terms of new creature animations for a reason. However, take heart in knowing that as slow as things seem here, it could and has been much worse. As I said, I was clueless in the use of most of this software when I first had the idea to try and add my own custom animations. This whole system was built through repeated trial-and-error over multiple attempts over the last few years, trying to find the optimal tools to work with. Weeks of wasted effort, multiple gigabytes worth of rendered images that were unusable for one reason or another. This method has proven to be the best for me to stick with, but there are undoubtedly many ways to improve upon it that I just haven't understood yet. If anyone more technically competent has any thoughts to share, I would gladly welcome it, because I feel this area of IE modding is something that could be very useful for modders if it were more accessible to them.

The intellect devourer animation, along with any other animations I may add to the repository in the future should I feel like it, is free to be used by anyone in the modding community as they please, no credit required. I consider this less of a personal project and more of a resource that I hope others may use and add to over time. It may be a stretch, but I hope that this may lead into something greater for IE mods in general. Thanks for reading, and I hope this has been useful to you and others.

Version history

7/8/2025 – Updated guide to explain use of newer and improved .blend file, using post-processing functions to render the object and shadow separately and removing alpha for better sprites

2/14/2025 – changed section on exporting textures to recommend higher resolution .dds files instead of .tga

2/13/2025 – initial version